

Efficient In-Context Learning for 3D Medical Image Segmentation

Submitted to the
Albert-Ludwigs-Universität Freiburg

by Tidiane Camaret Ndir



Master's Thesis

Efficient In-Context Learning for 3D Medical Image Segmentation

Submitted on **October 7, 2026**

to the **Albert-Ludwigs-Universität Freiburg**

by **Tidiane Camaret Ndir** (5364532) 

Examined by **Prof. Dr. Thomas Brox, Prof. Dr. Maxim Zaitsev**

Department of Computer Science

Faculty of Engineering

Albert-Ludwigs-Universität Freiburg

Declaration

I hereby declare, that I am the sole author and composer of my thesis and that no other sources or learning aids, other than those listed, have been used.

Furthermore, I declare that I have acknowledged the work of others by providing detailed references of said work.

I also hereby declare that my thesis has not been prepared for another examination or assignment, either in its entirety or excerpts thereof.

Freiburg, October 7, 2026

Place, date

Tidiane Camaret Ndir

Acknowledgements

I would like to thank Robin T. Schirrmeister, Marco Reisert and Elias Kellner for their support and collaboration.

Abstract

In-context segmentation predicts the mask of a target structure from a few annotated examples, so a single model can segment new classes without retraining. For 3D medical images, compute becomes a central constraint, since the cost of dense attention between target and context grows quadratically with the number of voxels. This thesis develops an efficient 3D in-context segmentation model around three questions: how to combine image and label information, where to spend compute, and how to obtain training tasks beyond the annotated classes.

First, we propose dual attention, which keeps image and label tokens in separate streams that interact in every layer. At matched compute and with fewer parameters, it outperforms merging the two once on 12 of 13 evaluation splits, with the largest gains on new targets. Second, we propose a coarse-to-fine cascade in which each level processes a single crop placed by the previous level's prediction, which also enters the target's label tokens as a prior. Training with the model's own predictions as prior outperforms training with perturbed ground-truth masks. Third, we propose a synthetic task generator that paints procedural shapes onto real scans; compared with a fully synthetic canvas, it improves three of four lesion datasets.

We evaluate the model on held-out datasets and compare it with Medverse. With matched refinement steps, our final model is 15 times faster and more accurate on average over the non-lesion datasets (Dice 0.615 vs. 0.552), while Medverse remains more accurate on two of them. Lesions remain difficult for both models and are the main open problem.

Zusammenfassung

In-Context-Segmentierung sagt die Maske einer Zielstruktur anhand weniger annotierter Beispiele vorher, sodass ein einzelnes Modell neue Klassen ohne erneutes Training segmentieren kann. Bei medizinischen 3D-Bildern wird der Rechenaufwand zur zentralen Einschränkung, da die Kosten dichter Attention zwischen Ziel und Kontext quadratisch mit der Anzahl der Voxel wachsen. Diese Arbeit entwickelt ein effizientes Modell zur 3D-In-Context-Segmentierung entlang dreier Fragen: wie Bild- und Labelinformation kombiniert werden sollte, wofür Rechenaufwand eingesetzt werden sollte und wie sich Trainingsaufgaben über die annotierten Klassen hinaus gewinnen lassen.

Erstens schlagen wir Dual Attention vor, bei der Bild- und Label-Tokens in getrennten Strömen verbleiben, die in jeder Schicht miteinander interagieren. Bei gleichem Rechenaufwand und mit weniger Parametern übertrifft sie eine einmalige Fusion beider Ströme in 12 von 13 Evaluationsgruppen, mit den größten Zugewinnen bei neuen Zielstrukturen. Zweitens schlagen wir eine Coarse-to-Fine-Kaskade vor, in der jede Stufe einen einzelnen Ausschnitt verarbeitet, der anhand der Vorhersage der vorherigen Stufe platziert wird; diese Vorhersage geht zudem als Prior in die Label-Tokens des Ziels ein. Das Training mit den eigenen Vorhersagen des Modells als Prior übertrifft das Training mit gestörten Ground-Truth-Masken. Drittens schlagen wir einen Generator synthetischer Aufgaben vor, der prozedurale Formen in reale Aufnahmen einzeichnet. Verglichen mit vollständig synthetischen Bildern als Untergrund verbessert er die Ergebnisse auf drei von vier Läsionsdatensätzen.

Wir evaluieren das Modell auf TotalSegmentator und neun weiteren, nicht im Training verwendeten Datensätzen und vergleichen es mit Medverse. Bei gleicher Anzahl an Verfeinerungsschritten ist unser finales Modell 15-mal schneller und im Mittel über die Datensätze ohne Läsionen genauer (Dice 0,615 gegenüber 0,552), während Medverse auf zwei dieser Datensätze genauer bleibt. Läsionen bleiben für beide Modelle schwierig und sind das zentrale offene Problem.

Contents

1	Motivation	1
2	Fundamentals	3
2.1	In-Context Segmentation	3
2.2	Volumetric Encoders and Attention	3
2.3	Synthetic Training Data	4
2.4	Related Work	4
3	Methodology	6
3.1	Problem Setup and Backbone	6
3.2	Dual Attention	7
3.3	Coarse-to-Fine Cascade	9
3.4	Synthetic Task Generation	10
3.5	Training Objective	11
4	Results	15
4.1	Experimental Setup	15
4.2	Image–Label Fusion	16
4.3	Coarse-to-Fine Cascade	18
4.4	Synthetic Task Generation	22
5	Discussion	28
5.1	Image–Label Fusion	28
5.2	Coarse-to-Fine Cascade	28
5.3	Synthetic Task Generation	29
5.4	Limitations	29
5.5	Open Questions	30
6	Conclusion	31
	List of Figures	I
	List of Tables	III
	List of Algorithms	V
	Appendix	VI

1 Motivation

In-context learning (ICL) treats segmentation as a few-shot matching problem. The model receives a target scan and a small set of context pairs, each an image with a mask of the structure of interest, and predicts the corresponding mask for the target without task-specific fine-tuning. A single trained model can therefore segment new anatomical classes without retraining. This is useful for structures that are rare, newly defined, or annotated at only a few sites.

Most ICL segmentation models were developed and evaluated on 2D images. CT and MRI are volumetric, however, and processing them slice by slice discards through-plane context that helps to resolve thin or elongated structures. Working directly on 3D volumes avoids this, but makes compute a central constraint. The number of tokens grows with the cube of the volume’s side length, and the cost of dense attention between target and context grows with the square of the number of tokens. Going from a 64^3 to a 256^3 volume multiplies the number of voxels by 64 and the cost of dense attention by roughly 4000.

This thesis addresses three questions that arise when building an efficient 3D ICL segmentation model.

1. **How should image and label information be combined?** In existing models, image and mask information is merged once, early in the network, either by concatenating channels before the encoder or through a single fusion operation. We instead keep image and label tokens as separate streams and let them exchange information at every layer through *dual attention* (subsection 3.2).
2. **Where should compute be spent?** Medverse [?] refines its prediction by processing the entire volume with a sliding window at every resolution step, including regions that the previous step has already ruled out, and passes the previous prediction through a separate network branch. We use a *coarse-to-fine cascade* in which each level processes only the region localized by the previous level, and the previous prediction enters the target’s label tokens directly as a prior (subsection 3.3).
3. **Where can training data come from beyond the annotated classes?** Real image–mask pairs cover only the classes and appearance range of the training cohort. We generate additional training tasks with a *synthetic task generator* that paints procedurally generated shapes with spatially correlated texture onto real scans, which

gives exact labels for structures outside the annotated classes (subsection 3.4).

Contributions.

- A dual attention mechanism that keeps image and label tokens separate. We compare it with a compute-matched variant that merges both into a single token per cell, and with a dual U-Net cross-attention model initialized from Medverse.
- A coarse-to-fine approach that passes each level’s prediction to the next level as a prior and restricts fine-level processing to the localized region. We compare predicted priors with perturbed ground-truth priors, and compare accuracy and latency with Medverse’s coarse-to-fine approach.
- A synthetic task generator that paints procedural shapes onto real scans. We compare it with a SynthSeg-style synthetic canvas and evaluate its effect on lesion datasets and on TotalSegmentator.
- An evaluation on TotalSegmentator and nine held-out datasets, covering lesions, new anatomical structures, a non-anatomical target region, and, for the fusion comparison, MRI as an unseen modality.

Outline. section 2 introduces in-context segmentation, the volumetric encoders and attention mechanisms the model builds on, and related methods. section 3 describes the model, section 4 evaluates each component, section 5 discusses the findings and their limitations, and section 6 concludes.

2 Fundamentals

2.1 In-Context Segmentation

Given a target image I^t and a context set $\{(I_k^c, L_k^c)\}_{k=1}^K$ of image–mask pairs showing the same class, in-context segmentation predicts a target mask $\hat{L}^t$ in a single forward pass, without gradient updates or class-specific parameters. This suits medical imaging for three reasons. First, annotation requires scarce domain expertise and is expensive. Second, new structures and pathologies are defined faster than large supervised datasets can be curated for them. Third, differences in acquisition protocols limit how well a closed-set model trained at one site transfers to another, whereas context examples can come from the target site itself.

2.2 Volumetric Encoders and Attention

Convolutional volumetric backbones. The 3D U-Net [?] architecture encodes a volume into a feature pyramid with a downsampling encoder and decodes it back to full resolution. Deeper stages have lower spatial resolution and more channels, while shallow stages keep spatial detail at a low channel count. Our model uses this split directly: the coarse, high-channel stages feed the attention layers, and the fine stages are passed to the decoder as skip connections (subsection 3.1). UNETR [?] is an earlier example of combining transformer attention with a convolutional decoder for 3D segmentation.

Attention. Scaled dot-product attention [?] computes, for each query, a weighted average of value vectors. The weights are given by the similarity between the query and each key:

$$\text{Attn}(\mathcal{Q}, \mathcal{K}, \mathcal{V}) = \text{softmax}\left(\frac{\mathcal{Q}\mathcal{K}^\top}{\sqrt{d}}\right) \mathcal{V}, \quad (1)$$

where d is the key dimension. Multi-head attention runs several attention operations in parallel on lower-dimensional projections and concatenates their outputs, so that different heads can capture different relations within the same input.

Position encoding. Attention is permutation-invariant, so positional information has to be added explicitly. Rotary position embedding (RoPE) [?] rotates each query and key vector by an angle proportional to its position before the dot product. The resulting attention score depends only on the relative position of the two tokens. In 3D, RoPE can be applied per axis: the head dimension is split into three parts, and each part is rotated according to the token’s coordinate along one spatial axis. If these coordinates are given in physical units, i.e. scaled by the voxel spacing, attention between tokens

from volumes of different resolution reflects their physical distance rather than their grid indices. Our coarse-to-fine approach relies on this property, since its levels operate at different spacings (subsection 3.3).

2.3 Synthetic Training Data

Domain randomization trains a model on procedurally varied synthetic data so it does not overfit to the narrow appearance distribution of a single acquisition protocol or training cohort. SynthSeg [?] randomizes the appearance of whole images to make a model independent of contrast. Synthetic tasks for in-context segmentation serve a different purpose: they add new target classes, while the anatomy and appearance around the target can remain realistic. We therefore compare a SynthSeg-style canvas with painting synthetic shapes onto real scans (subsection 3.4).

2.4 Related Work

Prior in-context segmentation models make different choices on the three questions raised in section 1. Table 1 summarizes them.

UniverSeg. UniverSeg [?] was one of the first in-context segmentation models for medical images. It operates on 2D images. Its CrossBlock combines target and context features with a CrossConv layer, a convolution applied to each concatenated target–context feature pair, at every scale of a U-Net-like network. It incorporates synthetic tasks (procedurally generated geometric shapes) alongside real medical datasets during training to improve zero-shot generalization.

Medverse. Medverse [?] uses two separate 3D U-Nets, one for the target and one for the context. The context branch receives image and mask concatenated as channels, while the target branch receives only the image. The two branches exchange features at several stages: the context queries the target in the encoder, and the target queries the context in the decoder. Following UniverSeg, it also relies on procedural synthetic shapes during training to expand task diversity.

Iris. Iris [?] encodes the context pairs once into a compact task embedding per class, using pixel shuffle operations in its context encoder. The target interacts with the context only through this embedding, via a single bidirectional cross-attention step in the mask decoder, and the embedding is reused across queries. Large volumes are processed via a sliding window approach, without refinement across scales.

Table 1: Design choices of 3D in-context segmentation models on the three questions of section 1.

	Image-label fusion	Coarse-to-fine prior	Synthetic tasks
Iris	Once, in the context encoder (pixel shuffle)	None (sliding window)	No
Medverse	Once, at the input (channel concatenation)	Previous prediction in a separate branch; ground truth in training	Procedural shapes
Ours	In every layer (dual attention)	Previous prediction in the label tokens; own prediction in training	Procedural shapes on real scans

Image-label fusion. All three models fuse image and label information once, before target and context interact: UniverSeg and Medverse concatenate each context image with its mask as input channels, and Iris merges them in its context encoder. From then on, label information is carried only in fused features. Dual attention instead keeps a separate label stream throughout (subsection 3.2).

Coarse-to-fine approaches. Deep Neural Patchworks [?] segments large volumes by stacking levels of increasing resolution. It is a supervised, closed-set method rather than an in-context one, but it is an early example of a coarse-to-fine cascade for 3D segmentation. Medverse brings a similar idea to in-context segmentation with next-scale autoregressive prediction, following next-scale image generation [?]. The first step predicts the whole volume at low resolution. Each later step doubles the resolution and processes the entire volume with a sliding window. The previous prediction is passed, together with the target image, through a third U-Net branch that shares weights with the context branch. During training, a downsampled ground-truth mask stands in for this prediction.

3 Methodology

3.1 Problem Setup and Backbone

We use the setting of subsection 2.1: given a target volume I^t and K context pairs (I_k^c, L_k^c) showing the same class, the model predicts the target mask $\hat{L}^t$. Prediction runs through a cascade of M levels with decreasing voxel spacing $r_1 > r_2 > \dots > r_M$. Each level predicts a mask, which is passed to the next level as a prior (subsection 3.3). Within a level, the model consists of a shared encoder, a stack of dual attention layers, and a decoder (Figure 1).

At every level, all volumes are resampled onto the same grid of 128^3 voxels, and only the voxel spacing changes between levels. A coarse level therefore covers a large field of view at low resolution, and a fine level a small field of view at high resolution: at 6 mm spacing the grid spans 768 mm per side, at 1.5 mm only 192 mm. During training, the spacing is sampled from a range rather than fixed to the levels used at inference (subsection 3.3).

Shared encoder. All $K + 1$ images of a task pass through the same encoder, a plain convolutional encoder in the style of nnU-Net [?] with four stages, trained from scratch with He initialization. The encoder sees only images; masks enter the model through the label tokens described below. The first two stages (0 and 1) are the fine stages: they keep their own resolution and serve as skip connections for the decoder (subsection 3.2). The last two stages (2 and 3) are the coarse stages. Their features are resampled to a common grid of $R \times R \times R$ cells with $R = 16$, giving $N = R^3 = 4096$ cells per volume, and concatenated along the channel dimension. Each cell therefore covers $p^3 = 8^3$ voxels of the input grid, with $p = 128/R$. The attention layers always operate on this fixed grid, while the decoder can still produce a mask at the full 128^3 resolution.

Tokens. Each cell i of each volume v is represented by two tokens of dimension $D = 768$. The image token is a linear projection of the concatenated coarse features at that cell,

$$z_{v,i}^{\text{img}} = W_{\text{img}} f_{v,i}, \quad (2)$$

where $f_{v,i}$ denotes the resampled stage-2 and stage-3 features of cell i . The label token embeds the part of the mask that falls into the cell. We define $m_i(\cdot)$ as the function that extracts and flattens the $p^3 = 512$ mask voxels of cell i from a mask array, and write $m_{v,i}$ for $m_i(\cdot)$ applied to volume v 's mask. We project $m_{v,i}$ linearly:

$$z_{v,i}^{\text{lbl}} = \text{LblEmbed}(m_{v,i}) = W_{\text{lbl}} m_{v,i} + b_{\text{lbl}}. \quad (3)$$

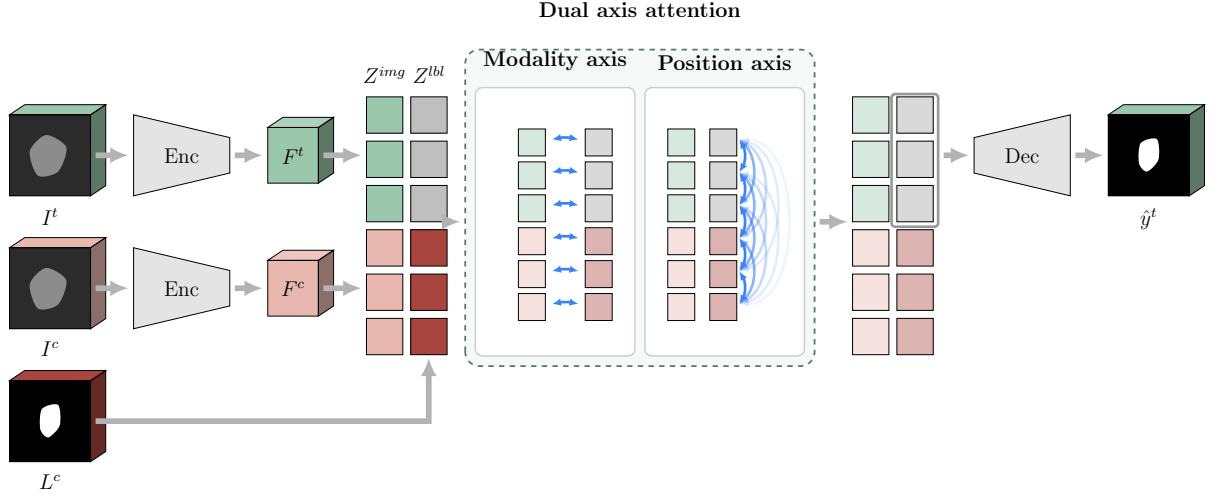


Figure 1: Overview of the model at one cascade level. Target and context images pass through a shared encoder; image and label tokens are processed by dual attention; a decoder produces the target mask.

We call the array supplying $m_{v,i}$ volume v 's *mask source*: whatever it is, the label token is formed the same way, by Equation 3. For a context volume, the mask source is the ground-truth mask L_k^c . For the target, the mask is unknown. At the first cascade level, every target cell receives the same placeholder as mask source: the average of $m_{v,i}$ over all cells of all context volumes, embedded with LblEmbed. It carries the average amount of foreground in the context masks, but no information about where the target is. At later levels, the target's mask source is the previous level's prediction instead (subsection 3.3).

Learned embeddings mark each token as belonging to the target or to a context volume. Eight learned register tokens, shared across all volumes, are prepended to the sequence. They are not tied to any cell and give the attention layers a fixed amount of global memory.

3.2 Dual Attention

The attention layers have two roles : combining image and label information, and matching the target against the context. In Medverse and Iris, these happen at fixed points in the network (subsection 2.4). Figure 2 contrasts Medverse's design with the shared-encoder design used in this thesis.

Token array. We arrange all tokens of a task in an array $Z \in \mathbb{R}^{S \times 2 \times D}$. Each of its S rows corresponds to one cell of one volume, and its two columns hold the image stream and the label stream (subsection 3.1). With $K+1$ volumes of N cells, $S = (K+1)N$. Dual attention attends along each of the two axes of this array in turn, following the general

pattern of the dual attention network of Fu et al. [?], which combines position-wise and channel-wise attention for scene segmentation.

Layer structure. Each of the $N_{\text{layer}} = 4$ layers applies three residual blocks, each preceded by RMS normalization:

$$\begin{aligned} Z &\leftarrow Z + \text{Attn}_{\text{mod}}(\text{RMSNorm}_1(Z)), \\ Z &\leftarrow Z + \text{Attn}_{\text{pos}}(\text{RMSNorm}_2(Z)), \\ Z &\leftarrow Z + \text{MLP}(\text{RMSNorm}_3(Z)). \end{aligned} \tag{4}$$

- **Modality axis** (Attn_{mod}). Within each row, the image token and the label token attend to each other. This is attention over a sequence of length two, computed independently for every row, so its cost grows linearly with S .
- **Position axis** (Attn_{pos}). Within each stream, the tokens of all rows attend to each other: image tokens attend to the image tokens of all cells and volumes, and label tokens likewise to label tokens. The two streams use the same attention weights but are processed as separate sequences of length S . Attention is unmasked, so every row attends to every other row, including context rows to target rows and vice versa. Queries and keys use 3D axial RoPE with physical coordinates (subsection 2.2), so attention depends on the physical distance between cells. The cost grows quadratically with S .
- **MLP**. Two linear layers with a GELU activation in between, applied to each token independently, with hidden dimension 3072.

All attention operations use 12 heads with a token dimension of $D = 768$.

The two streams exchange information only in the modality axis, which is local to a cell. Matching across cells and volumes happens within each stream. Because the modality axis comes first in every layer, the tokens entering the position axis already combine image and label information of their cell, so both streams can use both kinds of information when matching across cells. Since attention is unmasked, the context tokens also depend on the target: the context is encoded anew for each target, rather than once per task as in Iris.

Single axis attention baseline. As a baseline, the single axis attention variant (Figure 3, top) merges the image and label token of each cell into a single token before the

attention layers, using a pixel shuffle–unshuffle operation similar to the one Iris uses in its context encoder. The array then has a single column, and each layer applies only the position axis and the MLP. The image and label information of a cell is thus combined once, before any attention, rather than in every layer. Without the modality axis and the second stream, a layer costs roughly half as much, so this variant uses $N_{\text{layer}} = 9$ layers to match the compute of dual attention (subsection 4.2).

3.3 Coarse-to-Fine Cascade

Each of the M levels processes a single 128^3 crop; levels differ only in spacing and hence field of view. The prediction of one level determines where the next crop is placed and initializes the target’s label tokens.

Crop placement. The first crop is centered at the center of the volume, with a spacing chosen per dataset to cover the region where the target can occur. Each later crop is centered at the probability-weighted centroid of the previous prediction, or at the volume center if that prediction is empty. The finest spacing is chosen so that the field of view exceeds the largest expected target (Table 7). Each level thus costs one forward pass, independent of volume and target size. We refer to this as region restriction. Its drawbacks are that a target missed at a coarse level cannot be recovered, and that a target with several distant components may not fit into one fine crop.

Stitching. The coarsest prediction is pasted into the original volume first; each finer level then overwrites the region covered by its crop.

Prediction as prior. At level $\ell > 1$, the target’s mask source becomes the previous level’s prediction instead of the placeholder:

$$z_{t,i}^{\text{lbl}} = \text{LblEmbed}(m_i(\text{Warp}_{\ell-1 \rightarrow \ell}(\hat{y}_{\ell-1}^t))), \quad (5)$$

where t indexes the target volume, $m_i(\cdot)$ is the cell-extraction function of Equation 3, here applied to the warped prediction rather than to a stored per-volume mask, $\hat{y}_{\ell-1}^t$ is the sigmoid probability map of level $\ell-1$, detached from the computation graph, and Warp resamples it onto the crop grid of level ℓ . The prior thus enters the same stream as the context masks, instead of a separate branch as in Medverse (subsection 2.4); see Figure 4.

Training. All M levels are processed in one forward pass. The first crop is centered on the ground truth (at inference: on the volume center), later crops on the previous prediction, at its centroid or at a random foreground point. The coarsest spacing is

Table 2: Shape families. Target diameters are drawn from 10–50 mm unless stated otherwise.

Family	Family-specific parameters
Blob, disk	–
Splatter	2–6 components, spread 5–20 mm
Scatter field	10–120 components, spread 20–140 mm
Cylinder	length 30–100 mm
Vessel	trunk 40–120 mm, branching depth 2–4
Torus	ratio of major to minor radius 2–5

drawn per batch, and each finer spacing log-uniformly between the previous spacing and a lower bound, with a minimum ratio between levels. The prior is either the model’s own prediction (predicted prior) or a perturbed ground-truth mask (ground-truth prior); subsection 4.3 compares both.

3.4 Synthetic Task Generation

During training, a fraction p_{synth} of tasks is generated synthetically. For each synthetic task, one shape family and one set of shape and appearance parameters are drawn; these play the role of the class. Each of the $K + 1$ volumes then receives its own instance of the shape, painted onto its own image. Shape, size, and position vary between volumes with ratios of 0.3, 0.3, and 0.5, where 0 reuses the task-level value and 1 draws it independently.

Shapes. Masks come from seven procedural families (Table 2). The scatter field places many small components across a region to imitate multi-focal lesions, which few structures in the training cohort resemble. Sizes are defined in millimeters and converted to voxels at each cascade level, so a shape is the same physical object at every level.

Canvas. We compare two sources for the images. The *real-host* canvas uses real TotalSegmentator scans. A host structure is drawn, and $K + 1$ different subjects containing it are sampled; the shape is placed within ± 30 mm per axis of the host structure’s centroid. Since the volumes of a task are different scans, the target cannot be found by comparing it with the context images voxel by voxel. The *synthetic-bank* canvas uses TotalSegmentator training masks. A class is drawn, and $K + 1$ label maps containing it are selected, optionally preferring maps of similar geometry. As in SynthSeg [?], the image is synthesized from the label map with Gaussian-mixture intensities, and the shape is painted into it, with an optional blend of core and rim intensity. Only the anatomical layout of this canvas is real; its appearance is synthetic.

Appearance on the real-host canvas. The shape is painted relative to the local tissue statistics μ and σ of the host image:

$$I(x) = \mu + c \sigma + \sigma n(x), \quad c \sim \mathcal{U}(-2.5, 2.5), \quad (6)$$

with the contrast c drawn once per task. The texture $n(x)$ is fractal value noise with unit variance: four octaves of random grids, starting from 15 mm cells and halving the cell size per octave.

3.5 Training Objective

Each cascade level is supervised at its own resolution with the sum of a binary cross-entropy and a soft Dice loss:

$$\mathcal{L} = \sum_{\ell=1}^M \lambda_{\ell} (\mathcal{L}_{\text{BCE}}^{\ell} + \mathcal{L}_{\text{Dice}}^{\ell}), \quad (7)$$

with uniform weights $\lambda_{\ell} = 1$. Because the prior is detached (subsection 3.3), the loss at level ℓ does not propagate into level $\ell-1$. The single-level experiments of subsection 4.2 use $M = 1$.

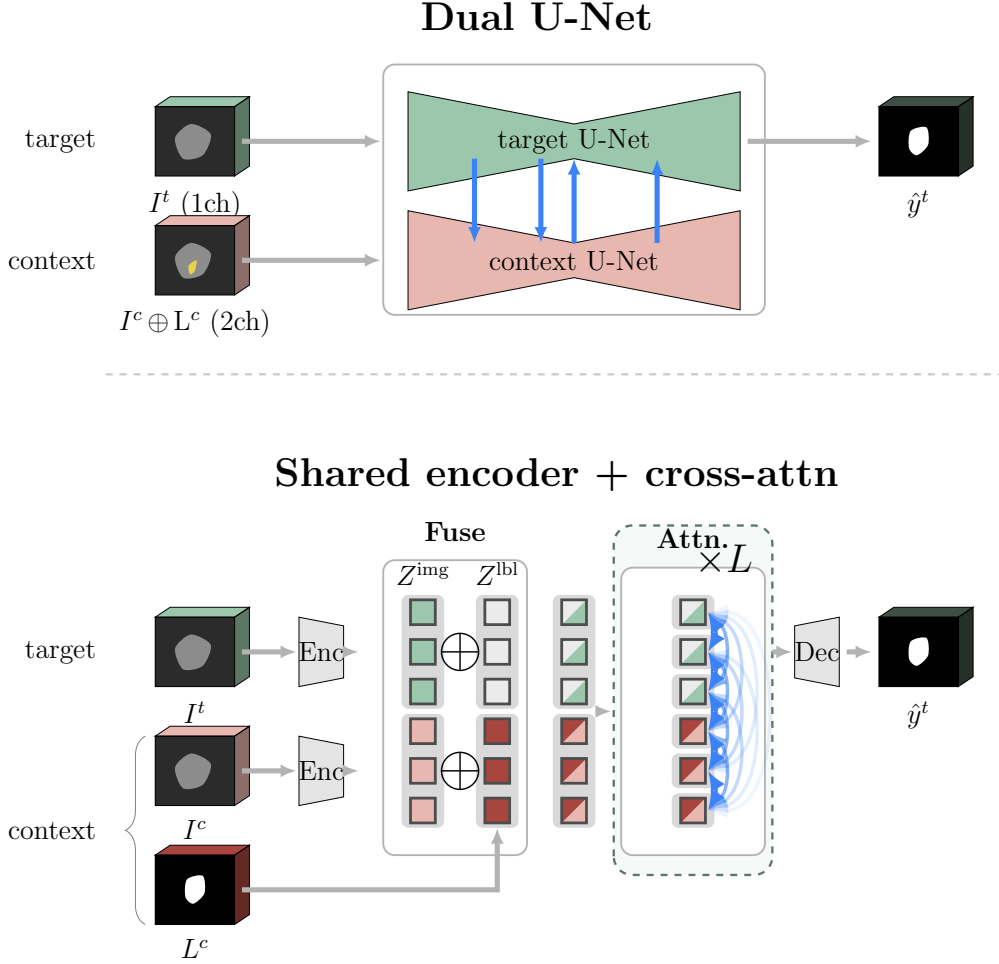
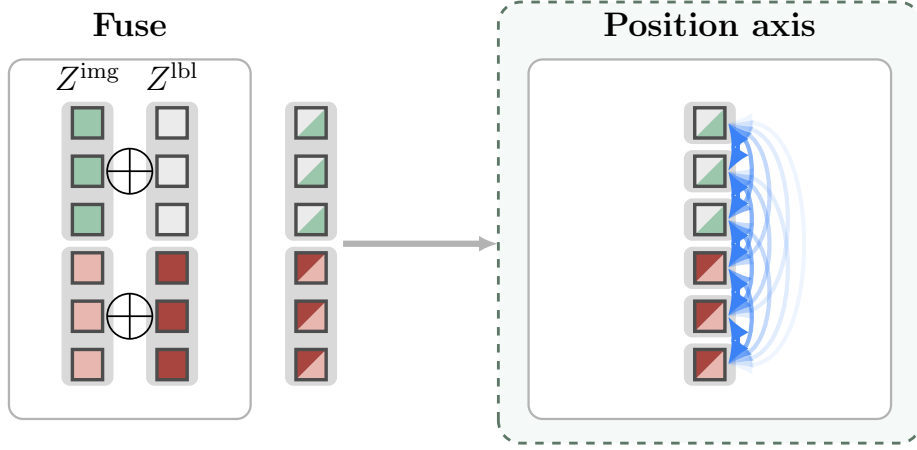


Figure 2: Two ways of coupling target and context, shown for one context volume ($K=1$) and three of its N tokens. Top: Medverse feeds the target image, and the context image and mask concatenated as channels, into two separate U-Nets that exchange features through cross-attention at several stages (blue arrows). Bottom: the shared-encoder design used in this thesis. Only images pass through the encoder. Label information is added to the encoded tokens afterwards, the tokens attend across cells and volumes for N_{layer} layers, and a shared decoder produces the mask. Figure 3 shows the two ways of adding label information that we compare.

Single axis attention



Dual axis attention

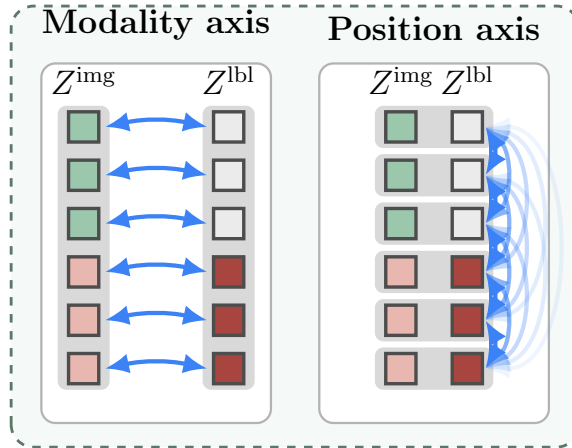


Figure 3: The two variants of the shared encoder design compared in subsection 4.2. Top: Single axis attention. Image and label tokens are merged once ($\oplus$), and the fused tokens attend across cells and volumes for N_{layer} layers. Bottom: Dual axis attention. Image and label tokens stay in separate streams. In each of the N_{layer} layers, the two tokens of a cell attend to each other (modality axis), and then each stream attends across all cells and volumes (position axis).

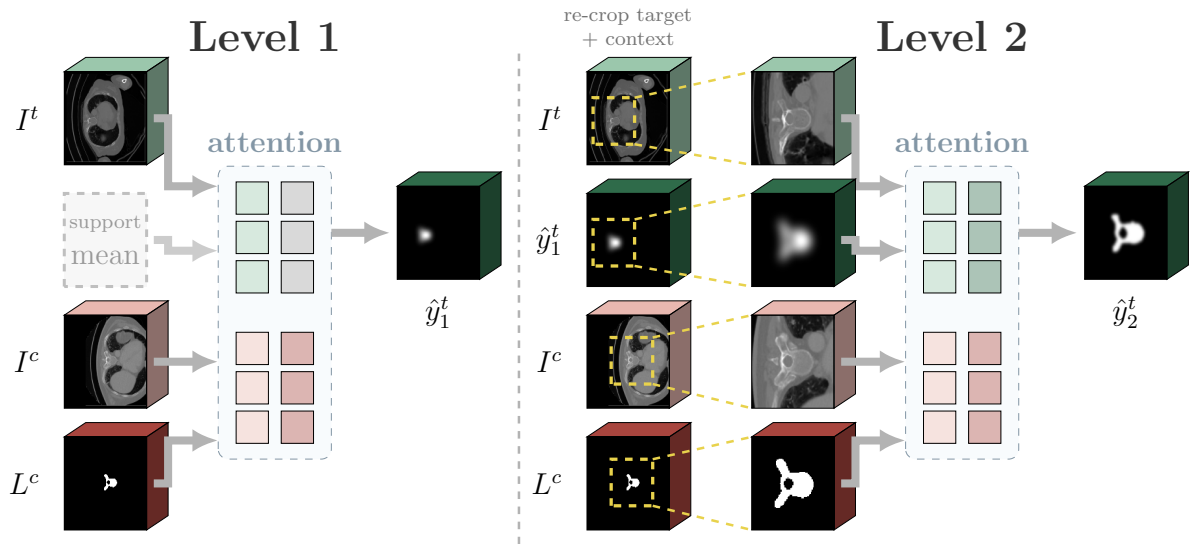


Figure 4: Label prior across two cascade levels. At level 1, the target's label tokens hold a placeholder (dashed); at level 2, a crop of the level-1 prediction $\hat{y}_1^t$.

4 Results

4.1 Experimental Setup

Training data. Models are trained on the CT part of TotalSegmentator [?] (1082 training, 57 validation, and 89 test subjects) and, except in subsection 4.2, on the TotalSegmentator MRI release [?] (561 training subjects from University Hospital Basel and the NIH Imaging Data Commons). Training uses 61 of the 117 classes, chosen to balance anatomical categories and structure thickness: 12 abdominal and pelvic organs, 8 thoracic, head, and spinal organs, 5 muscles (one side of each pair), 6 limb, shoulder, and pelvic bones, 10 vertebrae spanning all spine regions and the sacrum, 9 ribs and sternal structures, and 10 vessels. The remaining 56 classes form the unseen split (Table 15). It includes structures close to training classes, such as the contralateral side of each training muscle. On MRI, 31 of the 61 training classes are available.

Baselines. The external baseline is Medverse [?] with its released weights (subsection 4.3, subsection 4.4). In subsection 4.2, we additionally train Medverse’s architecture from its released weights with our training recipe.

Evaluation protocol. Each target is segmented with a single context pair ($K = 1$). The context subject is drawn at random, with a fixed seed per target, from the same dataset and split as the target, among the subjects that contain the target class. Every subject that contains a class serves once as a target for that class. For each model, we use the checkpoint with the best Dice on the in-distribution validation split; no evaluation dataset is used for checkpoint selection. Results of our models and of Dual U-Net Cross-Attn are means over three training seeds; the released Medverse weights are a single model.

Metrics. Accuracy is reported as Dice, computed per class and averaged over classes unless stated otherwise, and as Normalized Surface Dice (NSD) with a tolerance of 3 mm. Compute is reported as GFLOPs per forward pass and as latency per sample. All training and evaluation runs on a single NVIDIA RTX PRO 6000 Blackwell Server Edition GPU (96 GB). Means over datasets are weighted by each dataset’s number of evaluated tasks; unweighted means give the same ranking. In the cascade comparisons, the four lesion datasets are listed separately and excluded from the mean, since all models score below 0.1 Dice on them.

Datasets. Table 4 lists the evaluation datasets, grouped by their relation to the training classes. TotalSegmentator and FLARE22 contain classes from the TotalSegmentator vocabulary, so their classes split into seen and unseen ones. The other datasets contain

Table 3: Model configurations used in the results sections.

Experiment	Section	Training data	Levels	p_{synth}
Image–Label Fusion	4.2	TotalSeg CT	1	0
Cascade	4.3, 4.4	TotalSeg CT + MRI	2–3	0
Synth	4.4	TotalSeg CT + MRI	1	0.5
Synth + Cascade	4.4	TotalSeg CT + MRI	2–3	0.5

Table 4: Evaluation datasets, grouped by relation to the training classes. n : number of subjects; n_{cls} : number of evaluated classes.

Dataset	Modality	Target	n	n_{cls} (seen/unseen)
<i>Training vocabulary (seen and unseen classes)</i>				
TotalSegmentator [?]	CT	full-body anatomical structures	89	117 (61/56)
FLARE22 [?]	CT	abdominal organs	50	13 (11/2)
<i>New targets</i>				
ISLES22 [?]	MRI DWI	stroke lesion	250	1
Shifts-MS [?]	MRI FLAIR	MS lesions	46	1
MSD Hippocampus [?]	MRI T1	hippocampus	260	2
MSD Prostate [?]	MRI T2 + ADC	prostate zones	64	2
ATLAS v2.0 [?]	MRI T1	chronic stroke lesion	654	1
GNC_705	MRI Dixon	kidney lesions	610	13
NasalSeg [?]	CT	nasal cavity and sinuses	107	5
<i>Non-anatomical target</i>				
HU_LWK1	CT	center of L1 vertebra	36	1
<i>Additional modality (unseen in subsection 4.2)</i>				
TotalSeg MRI [?]	MRI	full-body anatomical structures	47	50 (31/19)

targets that do not occur in training: new anatomical structures, lesions, a measurement region that is not an anatomical object, and, for the fusion experiments, an unseen modality. Because of this split, "held-out" is used at two levels below: at the dataset level (a dataset whose images were never trained on, e.g. FLARE22, whose classes are nonetheless mostly in-vocabulary) and, where stated explicitly as such, at the class level (excluding classes that are in the training vocabulary, even if their dataset is held out).

4.2 Image–Label Fusion

This section compares three architectures. The comparison between Dual U-Net and shared-encoder architectures tests the change from two U-Nets coupled by cross-attention to a shared encoder with attention layers. The comparison between single axis and dual axis attention tests whether, within the shared-encoder design, keeping image and label tokens separate improves over merging them once.

Table 5: Training settings shared by the three architectures of subsection 4.2.

Optimizer	AdamW
Learning rate	10^{-4}
Schedule	cosine decay, 10-epoch warmup
Weight decay	10^{-2}
Batch size	8
Gradient clipping	1.0
Loss	BCE + Dice
Epochs	250

Architectures. *Dual U-Net Cross-Attn* is Medverse’s architecture (subsection 2.4), initialized from the released weights and then trained with our recipe. It is therefore a different model from the released Medverse baseline used in subsection 4.3. *Single axis* and *Dual axis attention* are the two variants shown in Figure 3. Early fusion uses $N_{\text{layer}}=9$ layers to match the compute of dual attention (subsection 3.2). This gives both variants similar compute (3973 vs. 3892 GFLOPs). Parameter count could not be matched at the same time, and early fusion has 52% more parameters. Dual U-Net Cross-Attn is not compute-matched to either variant.

Training. All three architectures use the same training recipe (Table 5), data pipeline, and augmentations. They are trained on the CT cases and training classes of TotalSegmentator, at a single spacing level randomly sampled in the range 1.5–6 mm. Dual U-Net Cross-Attn starts from Medverse’s weights. Early fusion and dual attention are randomly initialized.

Evaluation. All three architectures are evaluated at a single level with 3 mm spacing, with crops centered on the ground truth, so localization is not tested here. Table 6 reports the results.

Results. Replacing the two U-Nets with a shared encoder raises Dice by 0.16 to 0.23 on the seen and unseen classes of TotalSegmentator and FLARE22, and from 0.178 to 0.232 on the unseen modality TotalSeg MRI. On all eight datasets with targets outside the training vocabulary, however, single-axis attention is below Dual U-Net Cross-Attn.

Dual attention improves over single-axis attention on 12 of the 13 rows; the exception is the two unseen FLARE22 classes (-0.028). The gains are small on the in-vocabulary rows (at most 0.018) and largest on new targets: NasalSeg ($+0.086$), Shifts-MS ($+0.071$), MSD Hippocampus ($+0.062$), and MSD Prostate ($+0.042$).

Compared with Dual U-Net Cross-Attn, dual attention is better on the four in-vocabulary

Table 6: Dice of the three architectures at a single level (3 mm), with parameter count, compute, and latency.

	Dual U-Net	Single axis Attn	Dual Axis Attn
TotalSeg CT, seen	0.348	0.574	0.582
TotalSeg CT, unseen	0.221	0.408	0.426
FLARE22, seen	0.532	0.693	0.709
FLARE22, unseen	0.443	0.632	0.604
MSD Prostate	0.377	0.205	0.247
MSD Hippocampus	0.172	0.122	0.184
HU_LWK1	0.146	0.079	0.102
TotalSeg MRI	0.178	0.232	0.259
ISLES22	0.046	0.037	0.043
Shifts-MS	0.087	0.038	0.109
ATLAS v2.0	0.031	0.014	0.022
GNC_705	0.044	0.014	0.025
NasalSeg	0.434	0.272	0.358
Mean of 10 held-out sources	0.187	0.174	0.201
Parameters	71.1M	74.1M	48.7M
GFLOPs	2362.6	3973.1	3892.5
Latency per sample	73.2 ms	73.0 ms	72.5 ms

rows, TotalSeg MRI, MSD Hippocampus, and Shifts-MS, and worse on the other six new-target datasets, most clearly MSD Prostate (-0.130) and NasalSeg (-0.076). Its higher held-out mean (0.201 vs. 0.187) is thus driven by FLARE22 and TotalSeg MRI, whose classes are largely in the training vocabulary; on the remaining eight datasets, Dual U-Net Cross-Attn leads (unweighted mean 0.167 vs. 0.136). Part of this lead may come from its initialization with Medverse’s weights.

Dual attention has the fewest parameters (48.7M); its compute is similar to single-axis attention by construction and 65% higher than Dual U-Net Cross-Attn. Latency is nearly identical for all three (72.5 to 73.2 ms).

4.3 Coarse-to-Fine Cascade

This section evaluates the Cascade configuration (Table 3): how accuracy develops across levels, how well the coarse level localizes the target, how the cascade compares with Medverse in accuracy and latency, and which prior works better during training.

Accuracy across levels. Table 7 reports the Dice of each level, with each crop placed from the previous level’s prediction. Dice increases from level to level on every dataset, e.g. from 0.603 to 0.751 on FLARE22 and from 0.065 to 0.152 on HU_LWK1. The

Table 7: Dice of the Cascade model at each level, with crops placed using the previous level’s prediction, and Dice of the final stitched prediction.

Dataset	Spacings (mm)	Dice per level	Stitched
FLARE22	[6, 3, 2.5]	0.603 → 0.718 → 0.751	0.751
MSD Prostate	[3, 1.5, 0.75]	0.276 → 0.361 → 0.381	0.375
HU_LWK1	[6, 3, 1]	0.065 → 0.109 → 0.152	0.152
TotalSeg MRI	[6, 3, 1.5]	0.365 → 0.479 → 0.495	0.503
NasalSeg	[1.5, 0.8]	0.484 → 0.547	0.547

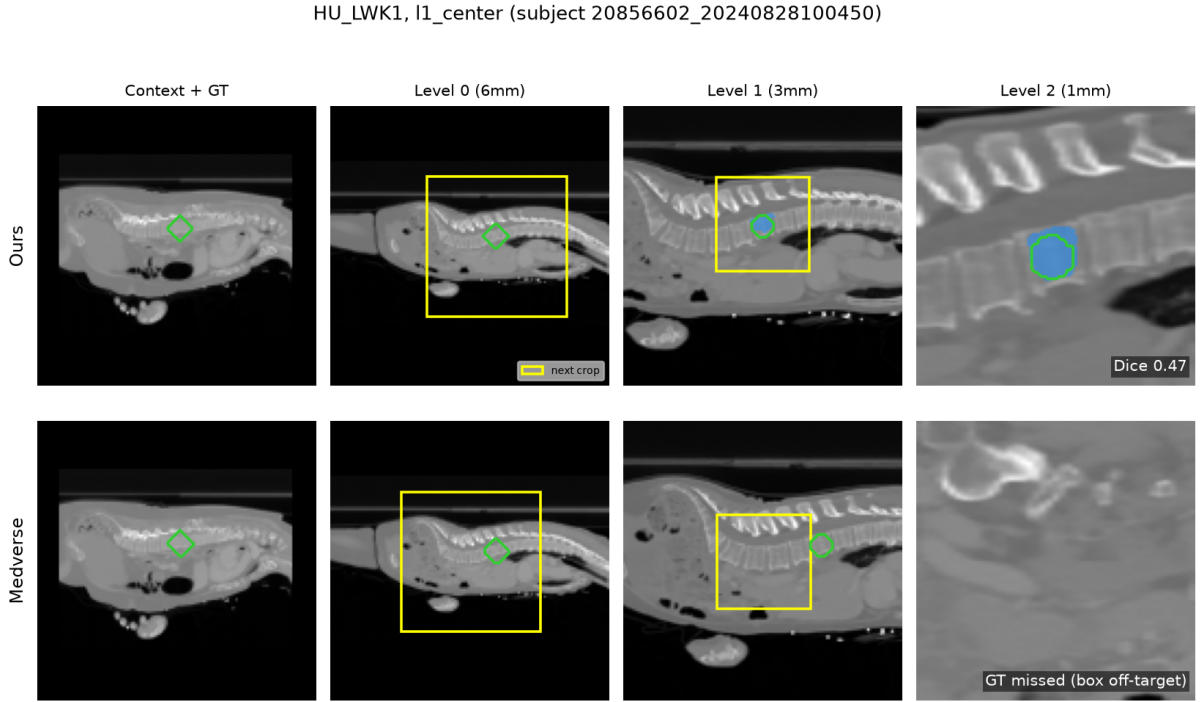


Figure 5: HU_LWK1: Cascade’s box tracks the vertebra center across all three levels; Medverse’s drifts off by level 1, so its finest crop no longer contains the target.

stitched mask keeps the coarser predictions outside the finest crop. This recovers target parts on TotalSeg MRI (0.495 to 0.503), adds false positives on MSD Prostate (0.381 to 0.375), and changes nothing where the coarser levels predict no foreground outside the finest crop.

Figure 5 and Figure 6 show the crop placement for Cascade and for Medverse run through the same cascade.

Localization. On TotalSeg CT, a middle-level crop centered at the centroid of the coarse prediction contains on average 92.2% of the target’s voxels, compared with 97.0% when centered at the ground-truth centroid. Localization thus loses about 5% of the target at this transition, and most of the remaining error arises within the fine crops. The middle-to-finest transition is not measured.

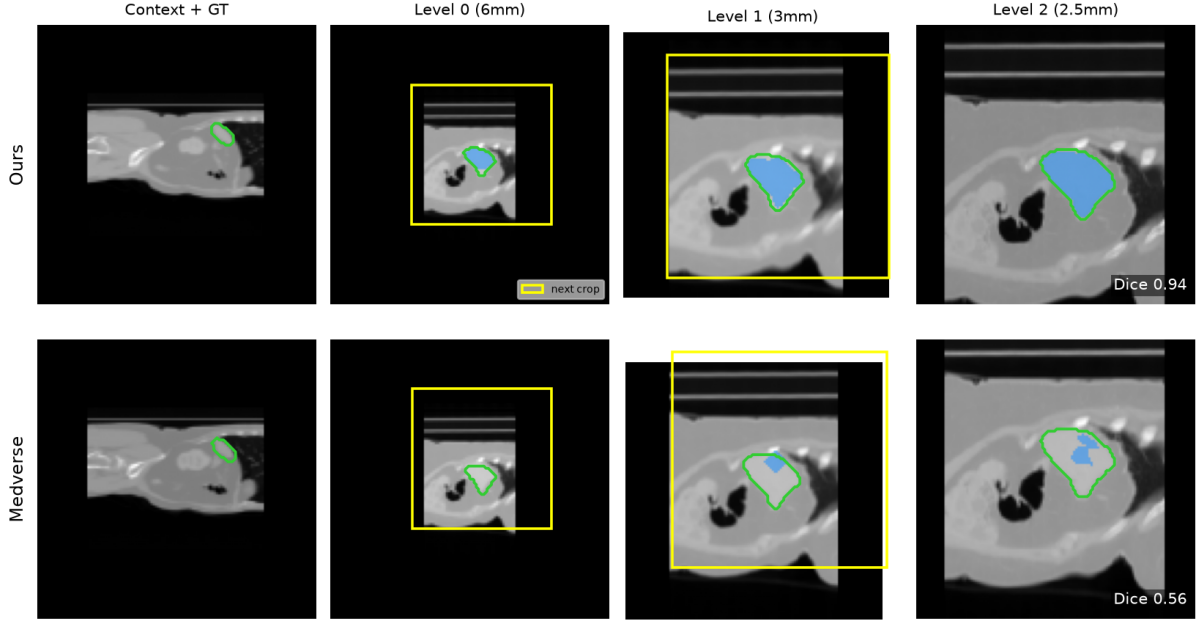


Figure 6: FLARE22: both models’ boxes converge on the spleen by level 1.

Comparison with Medverse. Table 8 compares Cascade with Medverse’s released weights, run with its own autoregressive inference and with its number of steps matched to our number of levels per dataset; its native depth is compared below. MSD Hippocampus runs at a single level, so its row compares the two models without a cascade. Medverse is not evaluated on TotalSeg CT.

On the six non-lesion datasets, Cascade is more accurate on four (FLARE22, MSD Prostate, HU_LWK1, TotalSeg MRI) and Medverse on two (MSD Hippocampus, NasalSeg); the sample-weighted means are 0.526 and 0.393. The largest gaps in our favor are on TotalSeg MRI, which is part of our training data, and on FLARE22, whose classes are mostly in our training vocabulary. Medverse’s two wins, in contrast, are on datasets that neither model saw during training. On the four lesion datasets, both models stay below 0.1 Dice.

Figure 7 and Figure 8 show one case from each side, with Medverse at its native depth. Ground truth is shown in green, Cascade in blue, and Medverse in orange. On a FLARE22 gallbladder, Cascade recovers the organ (Dice 0.79), while Medverse predicts a small, mislocated fragment (0.00). On MSD Hippocampus, Cascade’s prediction breaks into disconnected blobs (0.34), while Medverse’s is a single region close to the ground truth (0.70).

Table 8: Cascade vs. Medverse: macro Dice and mean latency per sample. Medverse is evaluated with its number of autoregressive steps matched to our number of levels per dataset.

Dataset	Spacings (mm)	Dice		Latency (ms)	
		Cascade	Medverse	Cascade	Medverse
FLARE22	[6, 3, 2.5]	0.751	0.359	311	4894
MSD Prostate	[3, 1.5, 0.75]	0.375	0.357	279	5006
MSD Hippocampus	[0.5]	0.303	0.686	99	50
HU_LWK1	[6, 3, 1]	0.152	0.020	439	5336
TotalSeg MRI	[6, 3, 1.5]	0.503	0.055	194	5188
NasalSeg	[1.5, 0.8]	0.547	0.737	161	1886
Mean (sample-weighted) [†]		0.526	0.393	206	3491
ISLES22	[3, 1.5]	0.069	0.059	155	1894
Shifts-MS	[3, 1.5]	0.031	0.098	289	1911
ATLAS v2.0	[3.8, 1.9]	0.026	0.024	149	1862
GNC_705	[6, 3, 1.2]	0.024	0.007	288	4874

[†]Over the six datasets above the second midrule, each weighted by its number of evaluated tasks (36 for HU_LWK1 to 650 for FLARE22). Unweighted: Dice 0.439 vs. 0.369, latency 247 vs. 3727 ms. The four lesion datasets are excluded, since both models score below 0.1 on all of them.

Cascade is about $17\times$ faster than step-matched Medverse (206 vs. 3491 ms per sample, sample-weighted; $15\times$ unweighted, 247 vs. 3727 ms). At a single level (MSD Hippocampus), Medverse is faster (50 vs. 99 ms). The speedup therefore comes from processing one crop per level instead of sliding a window over the whole volume at each step, not from a cheaper forward pass.

Medverse at native depth. By default, Medverse runs more autoregressive steps than our cascade has levels. Table 9 compares its Dice at both depths.

Matching lowers Medverse’s Dice on seven of the ten datasets, most on FLARE22 (-0.081) and MSD Prostate (-0.066), and raises it on the other three by at most 0.006. At native depth, Medverse overtakes Cascade on MSD Prostate (0.423 vs. 0.375), so the two models split the non-lesion datasets three to three, and it exceeds 0.1 on one lesion dataset, Shifts-MS (0.152). The accuracy comparison therefore depends on how Medverse is run. The latency advantage does not, since native depth only adds steps.

Medverse with our cascade. We also ran Medverse’s weights inside our cascade inference (subsection 3.3) on seven held-out datasets, with and without its coarse prediction passed on as a prior (Table 16). Here Medverse runs a multi-level cascade on every dataset, including MSD Hippocampus ([6, 3, 0.5] mm), so these numbers are not directly comparable with Table 8. The prior lowers Dice on six of the seven datasets,

FLARE22 gallbladder (subject FLARE22_Tr_0001)

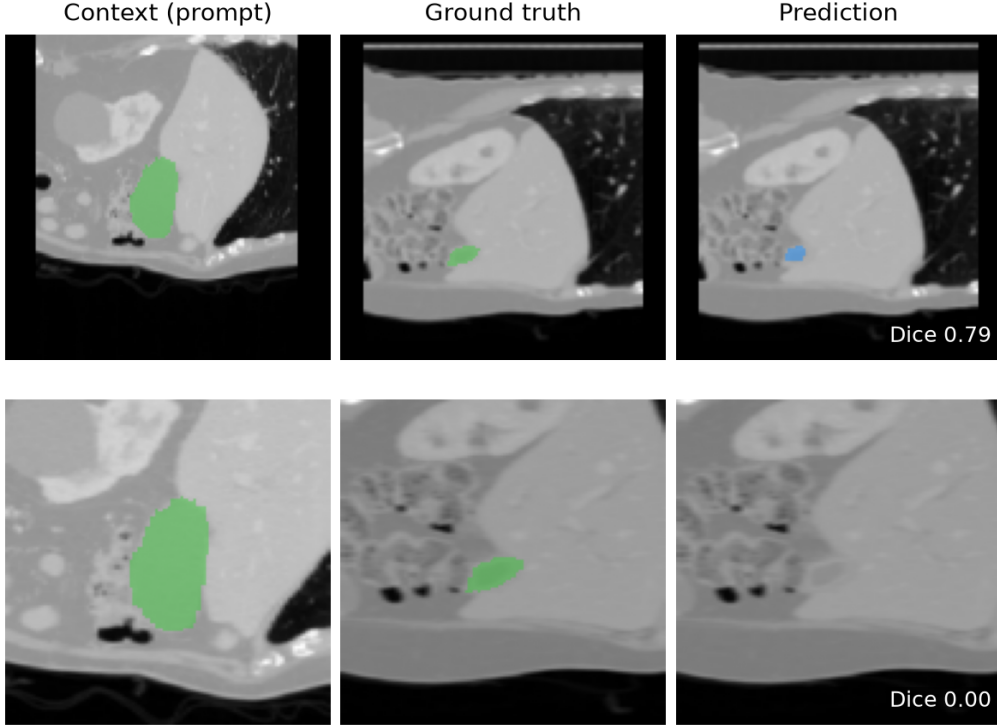


Figure 7: FLARE22 gallbladder, subject FLARE22_Tr_0001: Cascade recovers the organ, while Medverse (native depth) misses it almost entirely.

most on MSD Hippocampus (0.691 to 0.050) and ATLAS v2.0 (0.126 to 0.020); on HU_LWK1, Dice is 0 in both cases (Figure 5). Medverse is trained with a downsampled ground-truth mask as prior, so its own coarse errors are out of distribution for this input.

Source of the label prior. Table 10 compares two models that differ only in the prior used during training: the model’s own prediction from the previous level, or a perturbed ground-truth mask. Both use their own prediction at inference. The predicted prior is better on the three datasets from the training vocabulary; on TotalSeg CT, it is better on 99 of 117 classes (median +0.043) and raises NSD from 0.576 to 0.631. On the seven datasets outside the training vocabulary, it is better on MSD Prostate, HU_LWK1, and NasalSeg, by 0.053 to 0.148. On the four lesion datasets, both priors stay below 0.07 Dice and differ by at most 0.004.

4.4 Synthetic Task Generation

This section evaluates the synthetic task generator, first at a single level on the four lesion datasets it targets (ISLES22, Shifts-MS, ATLAS v2.0, GNC_705), then under cascade inference on all nine held-out datasets. The lesion datasets are the hardest in the evaluation: their targets are lesions on MRI with no counterpart among the training

MSD Hippocampus, posterior (subject hippocampus_001)

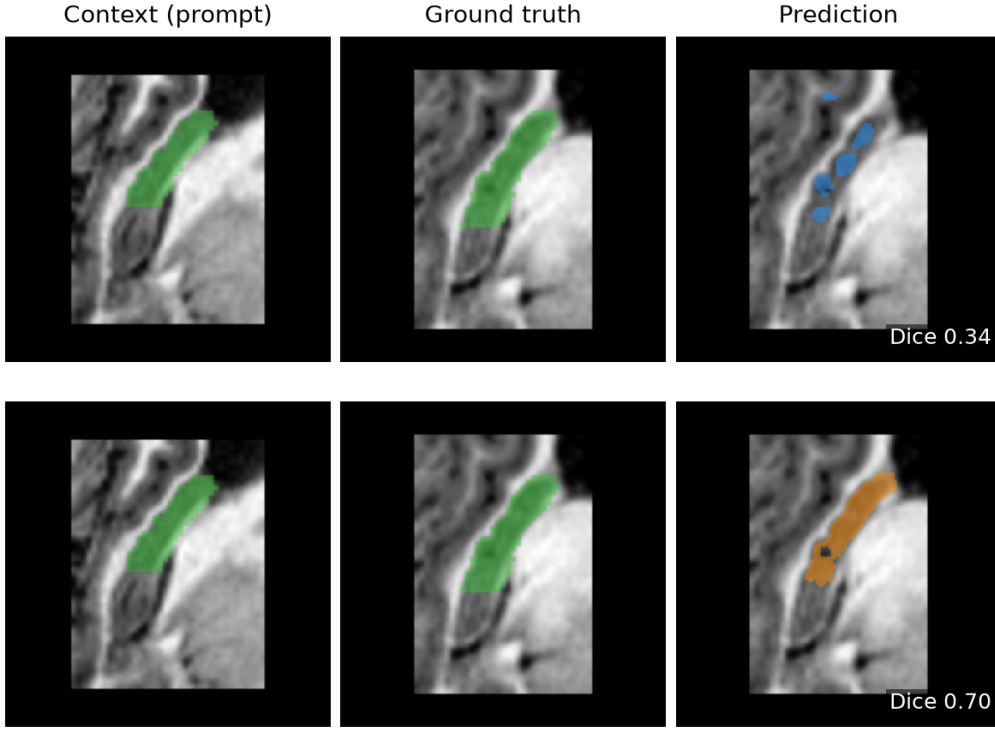


Figure 8: MSD Hippocampus, posterior class, subject hippocampus_001: Cascade’s prediction breaks into disconnected blobs, while Medverse (native depth) predicts a single contiguous region.

classes.

Synthetic-bank canvas. The baseline is the Synth configuration ($p_{\text{synth}} = 0.5$, synthetic-bank canvas). It reaches Dice between 0.008 and 0.082 on the four lesion datasets (Table 11). With this canvas, changing the mix of shape families or p_{synth} did not change these results beyond seed-to-seed variation. Since the bank’s images are synthesized from CT label maps, whereas all four lesion datasets are real MRI, we hypothesized that the canvas rather than the shapes limits transfer.

Real-host canvas. To test this, we continued training from the baseline in two runs that differ only in the canvas: the synthetic bank (stage 1) or real TotalSegmentator CT and MRI scans (stage 2). Both use only the scatter-field family, with the same p_{synth} and training budget. The real canvas improves Dice on three of the four datasets: ISLES22 (0.045 to 0.098), Shifts-MS (0.011 to 0.058), and GNC_705 (0.070 to 0.105). ATLAS v2.0 is unchanged (0.033 vs. 0.032). Absolute Dice stays below 0.11 on all four.

Stage 3 extends the real-host canvas to all seven shape families and to all TotalSegmentator subjects (CT and MRI, all organs), and trains longer. This raises GNC_705 to

Table 9: Medverse’s macro Dice with its number of autoregressive steps matched to our number of levels (matched, as in Table 8) and at its default number of steps (native).

Dataset	Spacings (mm)	Matched	Native
FLARE22	[6, 3, 2.5]	0.359	0.440
MSD Prostate	[3, 1.5, 0.75]	0.357	0.423
MSD Hippocampus	[0.5]	0.686	0.691
HU_LWK1	[6, 3, 1]	0.020	0.071
TotalSeg MRI	[6, 3, 1.5]	0.055	0.072
NasalSeg	[1.5, 0.8]	0.737	0.735
ISLES22	[3, 1.5]	0.059	0.053
Shifts-MS	[3, 1.5]	0.098	0.152
ATLAS v2.0	[3.8, 1.9]	0.024	0.023
GNC_705	[6, 3, 1.2]	0.007	0.011

Table 10: Macro Dice of Cascade models trained with a perturbed ground-truth prior or with their own predicted prior. Both use their own prediction as prior at inference.

Dataset	Ground-truth prior	Predicted prior
TotalSeg CT	0.543	0.585
TotalSeg MRI	0.449	0.503
FLARE22	0.731	0.751
MSD Prostate	0.310	0.375
HU_LWK1	0.099	0.152
NasalSeg	0.399	0.547
ISLES22	0.067	0.069
Shifts-MS	0.035	0.031
ATLAS v2.0	0.029	0.026
GNC_705	0.027	0.024

0.223, while the other three datasets fall slightly below their stage-2 values. Since stage 3 changes three factors at once, neither the gain nor the drops can be attributed to a single one.

Effect on TotalSegmentator. Table 12 reports TotalSegmentator validation Dice for the same checkpoints. Stages 1 and 2 stay within 0.022 of the baseline in every row, so the lesion gains of stage 2 do not cost accuracy on the training distribution. Stage 3 improves all four rows (mean 0.434 to 0.505), which likely reflects its longer training and larger canvas pool rather than the synthetic tasks themselves.

Cascade inference. Synth + Cascade continues the stage-3 checkpoint with 20 epochs of cascade training. Table 13 compares both with step-matched Medverse. Synth + Cascade improves over Cascade on MSD Hippocampus (0.303 to 0.518) and

Table 11: Single-level Dice on the lesion datasets for successive synthetic-task configurations. Stages 1 and 2 differ only in the canvas. Stage 3 adds all shape families, all TotalSegmentator subjects as canvas, and a longer training budget.

Dataset	Baseline	1) Synthetic bank	2) Real host, scatter	3) Real host, all families
ISLES22	0.050	0.045	0.098	0.087
Shifts-MS	0.008	0.011	0.058	0.051
ATLAS v2.0	0.028	0.033	0.032	0.027
GNC_705	0.082	0.070	0.105	0.223
Mean (sample-weighted) [†]	0.063	0.057	0.084	0.153

[†]Weighted by the number of evaluated tasks per dataset (46 for Shifts-MS to 1490 for GNC_705, which carries 61% of the weight). The unweighted mean (0.042, 0.040, 0.073, 0.097) gives the same ranking with a smaller stage-3 lead.

Table 12: TotalSegmentator validation Dice for the checkpoints of Table 11, by modality and class membership.

	Baseline	1) Synthetic bank	2) Real host, scatter	3) Real host, all families
TotalSeg CT, seen	0.479	0.476	0.474	0.572
TotalSeg CT, unseen	0.326	0.348	0.337	0.385
TotalSeg MRI, seen	0.505	0.500	0.498	0.590
TotalSeg MRI, unseen	0.395	0.379	0.379	0.430
Mean (sample-weighted) [†]	0.434	0.435	0.430	0.505

[†]Weighted by the number of evaluated tasks per row (155 to 273). Unweighted: 0.426, 0.426, 0.422, 0.494. Baseline and stage 1 are tied under either weighting.

NasalSeg (0.547 to 0.644), is worse on HU_LWK1 (-0.028) and Shifts-MS (-0.012), and stays within 0.011 elsewhere. These two gains put it ahead on the non-lesion mean (0.615, vs. 0.552 for Medverse and 0.531 for Cascade), although it still trails Medverse on both datasets. On the lesion datasets, the synthetic tasks have little effect under cascade inference: all three models stay below 0.1, and Medverse is best on Shifts-MS (0.098).

The stage-3 gain on GNC_705 (0.223 at a single level) is lost under cascade inference, where Synth + Cascade reaches 0.035. A likely cause is localization: small lesions that the coarse level misses fall outside the region-restricted fine crops and cannot be recovered (subsection 3.3). Figure 9 shows one such case: the lesion is a handful of voxels at the edge of the kidney, and the finest level predicts nothing there (Dice 0.00). Since the single-level results use crops centered on the ground truth, part of this drop reflects the loss of ground-truth localization.

Table 13: Dice on the held-out datasets under cascade inference. Cascade and Synth + Cascade use the same spacings per dataset; Medverse is step-matched as in Table 8.

Dataset	Spacings (mm)	Cascade	Synth + Cascade	Medverse
FLARE22	[6, 3, 2.5]	0.751	0.743	0.359
MSD Prostate	[3, 1.5, 0.75]	0.375	0.372	0.357
MSD Hippocampus	[0.5]	0.303	0.518	0.686
HU_LWK1	[6, 3, 1]	0.152	0.124	0.020
NasalSeg	[1.5, 0.8]	0.547	0.644	0.737
Mean (sample-weighted) [†]		0.531	0.615	0.552
ISLES22	[3, 1.5]	0.069	0.070	0.059
Shifts-MS	[3, 1.5]	0.031	0.019	0.098
ATLAS v2.0	[3.8, 1.9]	0.026	0.032	0.024
GNC_705	[6, 3, 1.2]	0.024	0.035	0.007

[†]Over the five datasets above the second midrule, weighted by the number of evaluated tasks (36 for HU_LWK1 to 650 for FLARE22). Unweighted: 0.426, 0.480, 0.432 (same ranking). The lesion datasets are excluded, since all three models score below 0.1 on all of them.

Final comparison: Synth + Cascade vs. Medverse. Table 14 adds NSD and latency for Synth + Cascade and step-matched Medverse. NSD agrees with Dice on which model is better on all nine datasets. On the non-lesion mean, Synth + Cascade leads in Dice (0.615 vs. 0.552) and NSD (0.729 vs. 0.631) and is $15\times$ faster (179 vs. 2697 ms per sample); Medverse is faster only on MSD Hippocampus, the one single-level dataset. Against Medverse at native depth (Table 9), Synth + Cascade wins only FLARE22 and HU_LWK1 among the non-lesion datasets, but stays ahead on the sample-weighted mean (0.615 vs. 0.586). Table 17 in the appendix gives per-class results.

GNC_705 hypo_mask_r (subject 100014_v30)

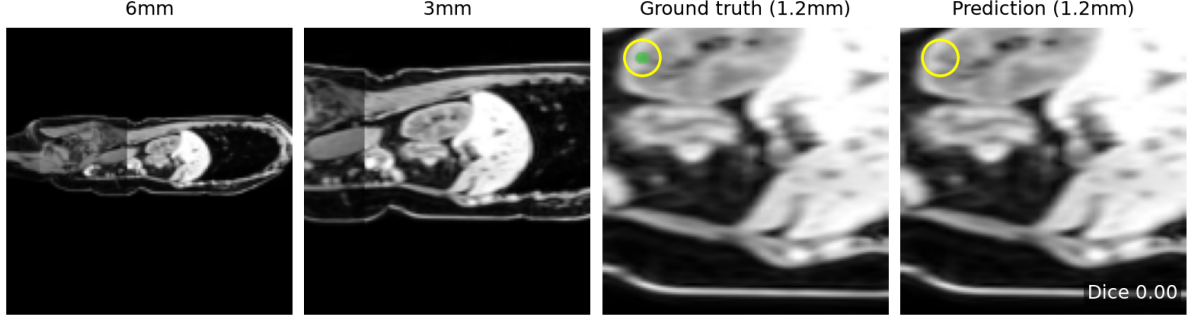


Figure 9: GNC_705, class hypo_mask_r, subject 100014_v30 (Cascade), with the coarse (6 mm) and middle (3 mm) levels shown for context. The lesion (yellow circle) is a handful of voxels at the edge of the kidney; the finest (1.2 mm) level predicts nothing there (Dice 0.00).

Table 14: Synth + Cascade (Ours) vs. step-matched Medverse (MV): Dice, NSD, and mean latency per sample on the nine held-out datasets. Bold marks the better value per row for Dice and NSD, and for latency only in the mean row.

Dataset	Spacings (mm)	Dice		NSD		Latency (ms)	
		Ours	MV	Ours	MV	Ours	MV
FLARE22	[6, 3, 2.5]	0.743	0.359	0.775	0.333	274	4894
MSD Prostate	[3, 1.5, 0.75]	0.372	0.357	0.314	0.300	278	5013
MSD Hippocampus	[0.5]	0.518	0.686	0.753	0.905	97	50
HU_LWK1	[6, 3, 1]	0.124	0.020	0.049	0.029	407	5336
NasalSeg	[1.5, 0.8]	0.644	0.737	0.791	0.844	106	1886
Mean (sample-weighted) [†]		0.615	0.552	0.729	0.631	179	2697
ISLES22	[3, 1.5]	0.070	0.059	0.086	0.070	145	1894
Shifts-MS	[3, 1.5]	0.019	0.098	0.123	0.136	272	1911
ATLAS v2.0	[3.8, 1.9]	0.032	0.024	0.044	0.012	151	1862
GNC_705	[6, 3, 1.2]	0.035	0.007	0.044	0.005	225	4874

[†]Over the five datasets above the second midrule, weighted by the number of evaluated tasks (36 for HU_LWK1 to 650 for FLARE22). Unweighted: Dice 0.480 vs. 0.432, NSD 0.536 vs. 0.482, latency 232 vs. 3436 ms. Dice and NSD are macro-averaged over classes; latency is a mean over samples.

5 Discussion

This chapter discusses the results for each of the three questions of section 1, the limitations of the evidence, and open questions.

5.1 Image–Label Fusion

Replacing Medverse’s two U-Nets with a shared encoder improved accuracy on the training vocabulary by 0.16 to 0.23 Dice, but not on new targets: there, single-axis attention was below Dual U-Net Cross-Attn on all eight datasets. Dual attention improved over single-axis attention on 12 of 13 rows, most on new targets. Letting image and label tokens interact in every layer thus seems to help most when the model has to extract the target from the context masks rather than recognize a trained class, although the results do not test this mechanism directly.

Against Dual U-Net Cross-Attn, dual attention led on the held-out mean, but this lead came from FLARE22 and TotalSeg MRI, whose classes are largely in the training vocabulary. On the eight datasets with new targets, Dual U-Net Cross-Attn was better on six. Its initialization from Medverse’s weights, trained on different data, may explain part of this; training both from the same initialization would separate the two. Dual attention reached its results with the fewest parameters of the three variants.

5.2 Coarse-to-Fine Cascade

Placing each crop from the previous level’s prediction works: Dice increases from level to level on every dataset (Table 7), and on TotalSeg CT the coarse prediction places the middle-level crop almost as well as the ground truth does (92% vs. 97% of the target’s voxels contained).

The clearest advantage over Medverse is speed: Cascade is more than an order of magnitude faster. Since Medverse is faster at a single level, this comes from processing one crop per level instead of the whole volume, not from the architecture. The accuracy comparison is less clear. On the non-lesion datasets, Cascade leads on four of six against step-matched Medverse but only on three of six at Medverse’s native depth. Its largest leads are on datasets from its training vocabulary, whereas Medverse leads on MSD Hippocampus and NasalSeg, which neither model saw during training.

Training with the model’s own prediction as prior was better than training with a perturbed ground-truth mask, both on the training vocabulary and on the three non-lesion datasets

outside it. A likely reason is that the model sees its own errors during training, which a hand-designed perturbation only approximates. Medverse, trained with a ground-truth prior, shows the opposite case: inside our cascade, passing on its own coarse prediction lowered its Dice on six of seven datasets. Whether a prior helps at all compared with none remains untested.

Region restriction makes the cost of each level independent of the volume size, but makes the fine levels depend on the coarse localization. On GNC_705, the single-level Dice of 0.223 fell to 0.035 under cascade inference, likely because the coarse level misses the small lesions. Small, multi-focal, or low-contrast targets are most at risk. Larger margins, several crops per volume, or a fallback to full-volume processing when the coarse prediction is uncertain could address this.

5.3 Synthetic Task Generation

With the synthetic-bank canvas, neither the mix of shape families nor p_{synth} affected the lesion datasets. Replacing the bank with real scans improved three of the four without reducing accuracy on TotalSegmentator. What the shapes are painted on thus mattered more than which shapes are painted. This change makes the canvas both real and, so the two factors are not separated.

Under cascade inference, Synth + Cascade improved mainly on MSD Hippocampus and NasalSeg, not on the lesion datasets that motivated the generator, and it differs from Cascade in more than the synthetic tasks. Lesion Dice stayed low throughout: at most 0.223 at a single level, and at most 0.070 for our models under cascade inference. Lesions vary in location, number, and shape far more than anatomical structures, so a single context example says little about where the target is. In-context lesion segmentation remains largely unsolved in this setting.

5.4 Limitations

Missing ablations. The cascade has not been evaluated without a prior or with full-volume processing at the fine levels, so the contributions of the prior and of region restriction are not isolated.

Evaluation. All results use a single context pair ($K = 1$), and the fusion comparison uses crops centered on the ground truth. Medverse is the only baseline evaluated empirically; Iris and UniverSeg are compared only by design. TotalSeg MRI is part of our training data, and Medverse’s training data may overlap with MSD Hippocampus

and NasalSeg, which affects the comparison on these datasets.

5.5 Open Questions

First, the position axis attends over all $K + 1$ volumes at once, so its cost grows quadratically with the number of context volumes; how accuracy and cost scale with K is untested. Second, region restriction needs a way to handle targets the coarse level cannot localize. Third, synthetic tasks did not yet help on lesions under cascade inference; tasks that resemble lesions in appearance and distribution, not only in shape, are one direction to explore.

6 Conclusion

This thesis studied in-context segmentation of 3D medical images and addressed three questions: how to combine image and label information, where to spend compute, and how to obtain training data beyond the annotated classes.

For the first question, we proposed dual attention, which keeps image and label tokens separate and lets them interact in every layer. Trained on TotalSegmentator CT, dual attention improved over merging the two once on 12 of 13 evaluation rows, most on new targets, and with the fewest parameters of the compared variants. A retrained Medverse architecture, initialized from its released weights, remained better on most datasets with new targets.

For the second question, we proposed a coarse-to-fine cascade in which each level receives the previous level’s prediction as a prior and processes only the region that prediction localizes. The cascade was more than an order of magnitude faster than Medverse, while the accuracy comparison depended on how Medverse was run. Training with the model’s own predictions as prior worked better than training with perturbed ground-truth masks.

For the third question, we proposed a synthetic task generator that paints procedural shapes onto scans. The canvas mattered more than the variety of shapes: painting onto real scans improved three of four lesion datasets at a single level without reducing accuracy on TotalSegmentator. Under cascade inference, however, lesion segmentation remained largely unsolved.

List of Figures

1	Overview of the model at one cascade level. Target and context images pass through a shared encoder; image and label tokens are processed by dual attention; a decoder produces the target mask.	7
2	Two ways of coupling target and context, shown for one context volume ($K=1$) and three of its N tokens. Top: Medverse feeds the target image, and the context image and mask concatenated as channels, into two separate U-Nets that exchange features through cross-attention at several stages (blue arrows). Bottom: the shared-encoder design used in this thesis. Only images pass through the encoder. Label information is added to the encoded tokens afterwards, the tokens attend across cells and volumes for N_{layer} layers, and a shared decoder produces the mask. Figure 3 shows the two ways of adding label information that we compare.	12
3	The two variants of the shared encoder design compared in subsection 4.2. Top: Single axis attention. Image and label tokens are merged once ($\oplus$), and the fused tokens attend across cells and volumes for N_{layer} layers. Bottom: Dual axis attention. Image and label tokens stay in separate streams. In each of the N_{layer} layers, the two tokens of a cell attend to each other (modality axis), and then each stream attends across all cells and volumes (position axis).	13
4	Label prior across two cascade levels. At level 1, the target's label tokens hold a placeholder (dashed); at level 2, a crop of the level-1 prediction $\hat{y}_1^t$.	14
5	HU_LWK1: Cascade's box tracks the vertebra center across all three levels; Medverse's drifts off by level 1, so its finest crop no longer contains the target.	19
6	FLARE22: both models' boxes converge on the spleen by level 1.	20
7	FLARE22 gallbladder, subject FLARE22_Tr_0001: Cascade recovers the organ, while Medverse (native depth) misses it almost entirely.	22
8	MSD Hippocampus, posterior class, subject hippocampus_001: Cascade's prediction breaks into disconnected blobs, while Medverse (native depth) predicts a single contiguous region.	23
9	GNC_705, class hypo_mask_r, subject 100014_v30 (Cascade), with the coarse (6 mm) and middle (3 mm) levels shown for context. The lesion (yellow circle) is a handful of voxels at the edge of the kidney; the finest (1.2 mm) level predicts nothing there (Dice 0.00).	27
10	HU_LWK1, example 1.	IX
11	HU_LWK1, example 2.	IX
12	ISLES22, example 1.	X
13	ISLES22, example 2.	X

14	Shifts-MS, example 1.	XI
15	Shifts-MS, example 2.	XI
16	MSD Hippocampus, example 1.	XII
17	MSD Hippocampus, example 2.	XII
18	MSD Prostate, example 1.	XIII
19	MSD Prostate, example 2.	XIII
20	ATLAS v2.0, example 1.	XIV
21	ATLAS v2.0, example 2.	XIV
22	GNC_705, example 1.	XV
23	GNC_705, example 2.	XV
24	FLARE22, example 1.	XVI
25	FLARE22, example 2.	XVI
26	NasalSeg, example 1.	XVII
27	NasalSeg, example 2.	XVII

List of Tables

1	Design choices of 3D in-context segmentation models on the three questions of section 1.	5
2	Shape families. Target diameters are drawn from 10–50 mm unless stated otherwise.	10
3	Model configurations used in the results sections.	16
4	Evaluation datasets, grouped by relation to the training classes. n : number of subjects; n_{cls} : number of evaluated classes.	16
5	Training settings shared by the three architectures of subsection 4.2. . .	17
6	Dice of the three architectures at a single level (3 mm), with parameter count, compute, and latency.	18
7	Dice of the Cascade model at each level, with crops placed using the previous level’s prediction, and Dice of the final stitched prediction. . . .	19
8	Cascade vs. Medverse: macro Dice and mean latency per sample. Medverse is evaluated with its number of autoregressive steps matched to our number of levels per dataset.	21
9	Medverse’s macro Dice with its number of autoregressive steps matched to our number of levels (matched, as in Table 8) and at its default number of steps (native).	24
10	Macro Dice of Cascade models trained with a perturbed ground-truth prior or with their own predicted prior. Both use their own prediction as prior at inference.	24
11	Single-level Dice on the lesion datasets for successive synthetic-task configurations. Stages 1 and 2 differ only in the canvas. Stage 3 adds all shape families, all TotalSegmentator subjects as canvas, and a longer training budget.	25
12	TotalSegmentator validation Dice for the checkpoints of Table 11, by modality and class membership.	25
13	Dice on the held-out datasets under cascade inference. Cascade and Synth + Cascade use the same spacings per dataset; Medverse is step-matched as in Table 8.	26
14	Synth + Cascade (Ours) vs. step-matched Medverse (MV): Dice, NSD, and mean latency per sample on the nine held-out datasets. Bold marks the better value per row for Dice and NSD, and for latency only in the mean row.	27

17	Per-class Dice and NSD, Synth + Cascade (ours) vs. Medverse, extending the dataset-level results of Table 14 to every class on all nine held-out datasets. Same protocol: Medverse’s number of autoregressive steps matched to our number of levels per dataset (subsection 4.3). Bold marks the better value per row; exact ties are left unbolded (GNC_705, Hyper mask L, NSD: both 0.000). MV: Medverse.	VI
15	TotalSegmentator classes used for training (seen) and held out (unseen), with the number of classes per category in parentheses. [†] : also available on TotalSegmentator MRI. On MRI, 50 classes are available, 31 of them seen.	VIII
16	Medverse’s released weights run inside our cascade inference (subsection 4.3, “Medverse with our cascade”), with and without its own coarse prediction passed forward as a prior. Macro Dice.	XVIII

List of Algorithms

Appendix

Table 17: Per-class Dice and NSD, Synth + Cascade (ours) vs. Medverse, extending the dataset-level results of Table 14 to every class on all nine held-out datasets. Same protocol: Medverse’s number of autoregressive steps matched to our number of levels per dataset (subsection 4.3). Bold marks the better value per row; exact ties are left unbolded (GNC_705, Hyper mask L, NSD: both 0.000). MV: Medverse.

Dataset	Class	Dice		NSD	
		Ours	MV	Ours	MV
FLARE22	Adrenal gland L	0.623	0.001	0.806	0.000
	Adrenal gland R	0.260	0.001	0.415	0.000
	Aorta	0.878	0.454	0.929	0.461
	Duodenum	0.500	0.020	0.502	0.029
	Esophagus	0.678	0.004	0.747	0.005
	Gallbladder	0.757	0.019	0.797	0.017
	Inferior vena cava	0.808	0.119	0.813	0.112
	Kidney L	0.910	0.903	0.919	0.875
	Kidney R	0.908	0.915	0.910	0.902
	Liver	0.950	0.945	0.893	0.851
	Pancreas	0.616	0.077	0.624	0.063
	Spleen	0.934	0.755	0.960	0.720
	Stomach	0.832	0.448	0.757	0.294
MSD Prostate	Prostate PZ	0.188	0.075	0.245	0.121
	Prostate TZ	0.557	0.640	0.382	0.480
MSD Hippocampus	Hippocampus anterior	0.562	0.704	0.767	0.902
	Hippocampus posterior	0.473	0.668	0.739	0.907
HU_LWK1	L1 center	0.124	0.020	0.049	0.029
NasalSeg	Maxillary sinus L	0.763	0.861	0.826	0.912
	Maxillary sinus R	0.799	0.822	0.864	0.890
	Nasal cavity L	0.419	0.633	0.685	0.805
	Nasal cavity R	0.443	0.556	0.737	0.740
	Nasopharynx	0.795	0.812	0.843	0.874
ISLES22	Stroke lesion	0.070	0.059	0.086	0.070
Shifts-MS	MS lesion	0.019	0.098	0.123	0.136
ATLAS v2.0	Stroke lesion	0.032	0.024	0.044	0.012

continued on next page

Table 17 – continued from previous page

Dataset	Class	Dice		NSD	
		Ours	MV	Ours	MV
GNC_705	Complex L	0.092	0.026	0.106	0.027
	Complex R	0.032	0.010	0.034	0.009
	Complex cyst L	0.038	0.004	0.050	0.003
	Complex cyst R	0.019	0.002	0.032	0.001
	Hyper L	0.053	0.003	0.075	0.001
	Hyper R	0.021	0.003	0.029	0.000
	Hyper cyst L	0.079	0.006	0.102	0.001
	Hyper cyst R	0.008	0.005	0.015	0.000
	Hyper mask L	0.000	0.004	0.000	0.000
	Hypo L	0.035	0.005	0.035	0.003
	Hypo R	0.016	0.009	0.018	0.005
	Hypo mask L	0.033	0.008	0.038	0.004
	Hypo mask R	0.033	0.010	0.036	0.008

Additional qualitative examples

Two additional context/ground-truth/prediction examples (Ours vs. Medverse) per held-out dataset, beyond the ones in subsection 4.3.

Table 15: TotalSegmentator classes used for training (seen) and held out (unseen), with the number of classes per category in parentheses. †: also available on TotalSegmentator MRI. On MRI, 50 classes are available, 31 of them seen.

Category	Seen	Unseen
Organs, abdomen and pelvis (12/5)	adrenal gland L†, colon†, duodenum†, esophagus†, gallbladder†, kidney L†, liver†, pancreas†, small bowel†, spleen†, stomach†, urinary bladder†	adrenal gland R†, kidney R†, prostate†, kidney cyst L, kidney cyst R
Organs, thorax, head, and spine (8/3)	brain†, heart†, spinal cord†, thyroid gland, trachea, lung lower lobe L, lung middle lobe R, lung upper lobe R	lung lower lobe R, lung upper lobe L, left atrial appendage
Muscles (5/5)	autochthon L†, gluteus maximus L†, gluteus medius R†, gluteus minimus L†, iliopsoas R†	autochthon R†, gluteus maximus R†, gluteus medius L†, gluteus minimus R†, iliopsoas L†
Bones, limbs, shoulder, and pelvis (6/5)	clavicle R†, femur L†, hip R†, humerus L†, scapula L†, skull	clavicle L†, femur R†, hip L†, humerus R†, scapula R†
Bones, spine (11/15)	sacrum†; vertebrae C3, C7, T3, T6, T9, T12, L1, L3, L5, S1	vertebrae C1, C2, C4–C6, T1, T2, T4, T5, T7, T8, T10, T11, L2, L4
Bones, ribs and sternum (9/17)	sternum, costal cartilages; ribs L2, L3, L9, L12, R2, R6, R11	ribs L1, L4–L8, L10, L11, R1, R3–R5, R7–R10, R12
Vessels (10/6)	aorta†, inferior vena cava†, portal and splenic vein†, iliac artery L†, iliac vein L†, superior vena cava, pulmonary vein, brachiocephalic trunk, common carotid artery L, subclavian artery L	iliac artery R†, iliac vein R†, common carotid artery R, subclavian artery R, brachiocephalic vein L, brachiocephalic vein R
MRI only (0/4)	–	lung L, lung R, vertebrae (all levels merged), intervertebral discs

HU_LWK1, l1_center: Ours vs. Medverse (subject 20856602_20240828100450)

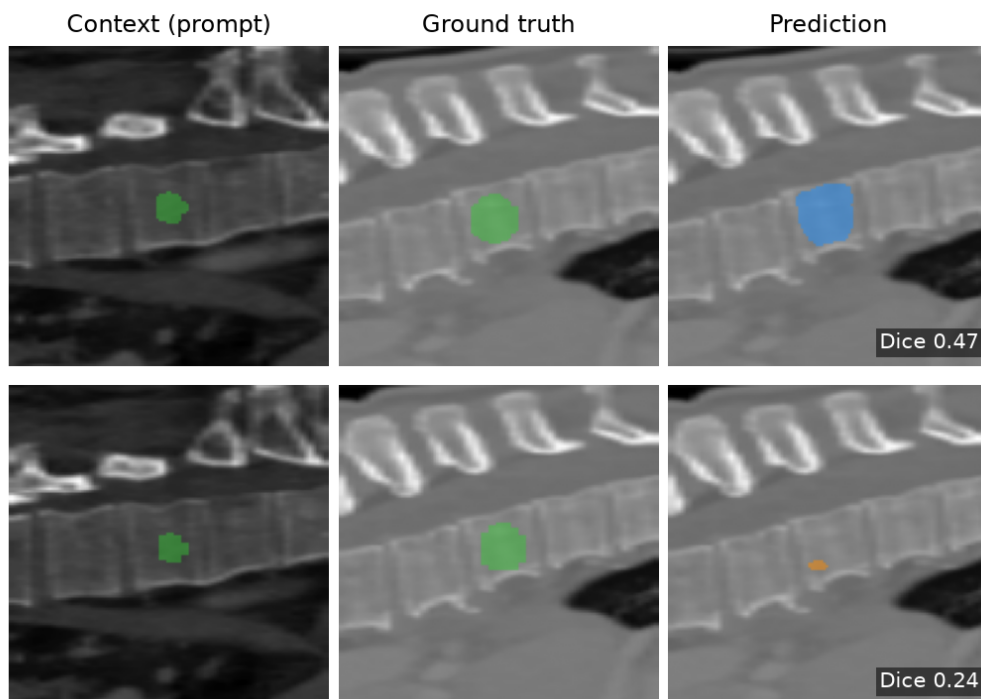


Figure 10: HU_LWK1, example 1.

HU_LWK1, l1_center: Ours vs. Medverse (subject 27441734_20240726125039)

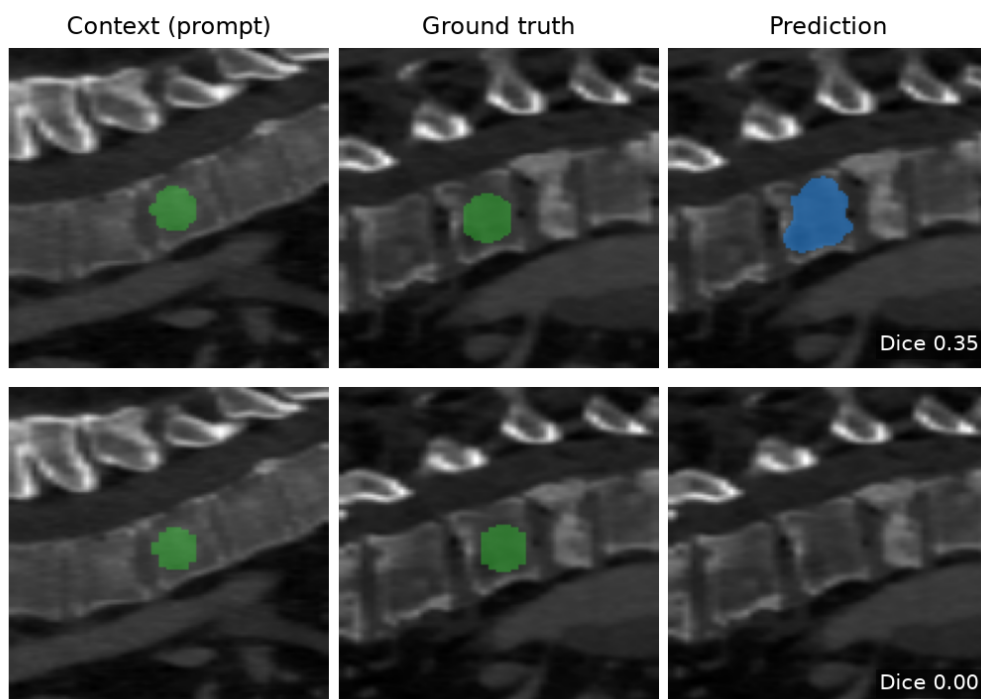


Figure 11: HU_LWK1, example 2.

ISLES22, stroke_lesion: Ours vs. Medverse (subject sub-strokecase0066)

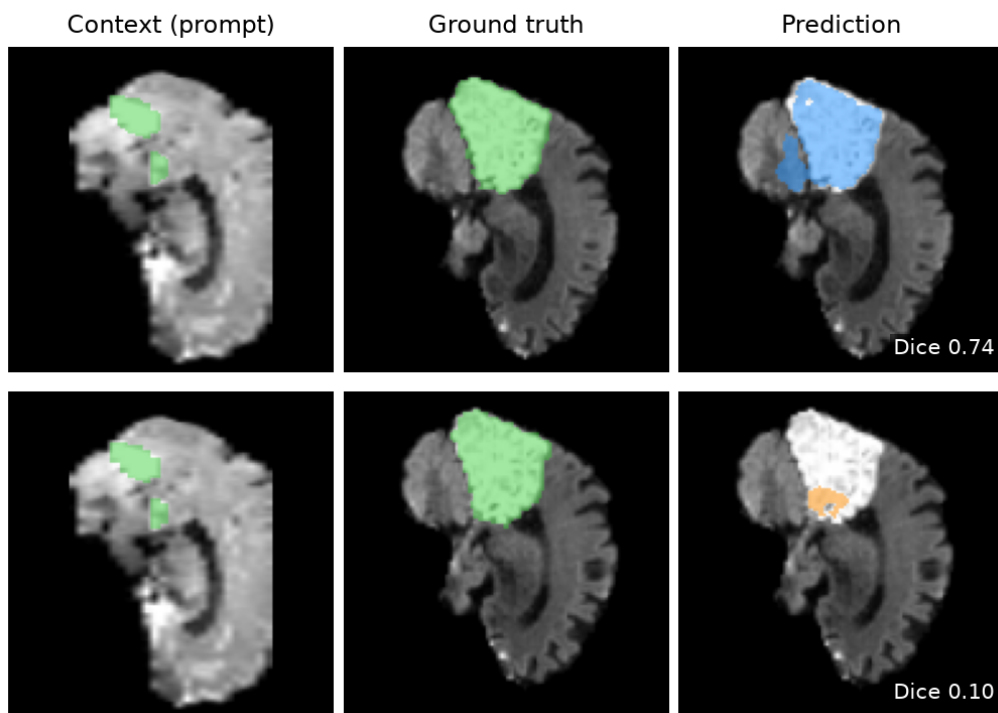


Figure 12: ISLES22, example 1.

ISLES22, stroke_lesion: Ours vs. Medverse (subject sub-strokecase0168)

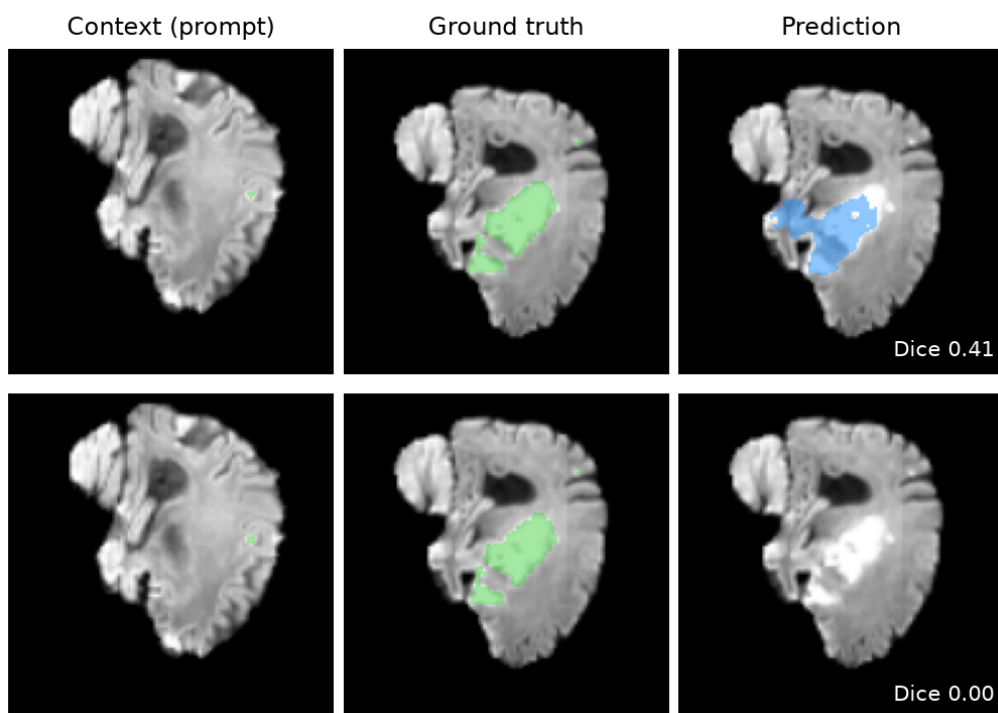


Figure 13: ISLES22, example 2.

Shifts-MS, ms_lesion: Ours vs. Medverse (subject ljubljana_dev_out_0021)

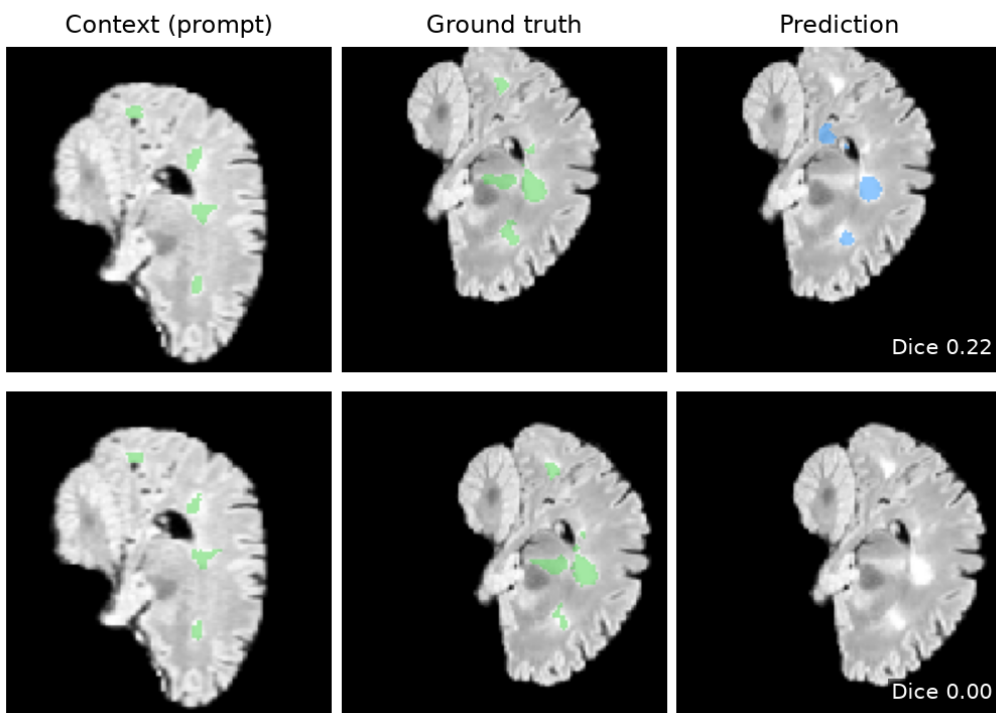


Figure 14: Shifts-MS, example 1.

Shifts-MS, ms_lesion: Ours vs. Medverse (subject ljubljana_dev_out_0008)

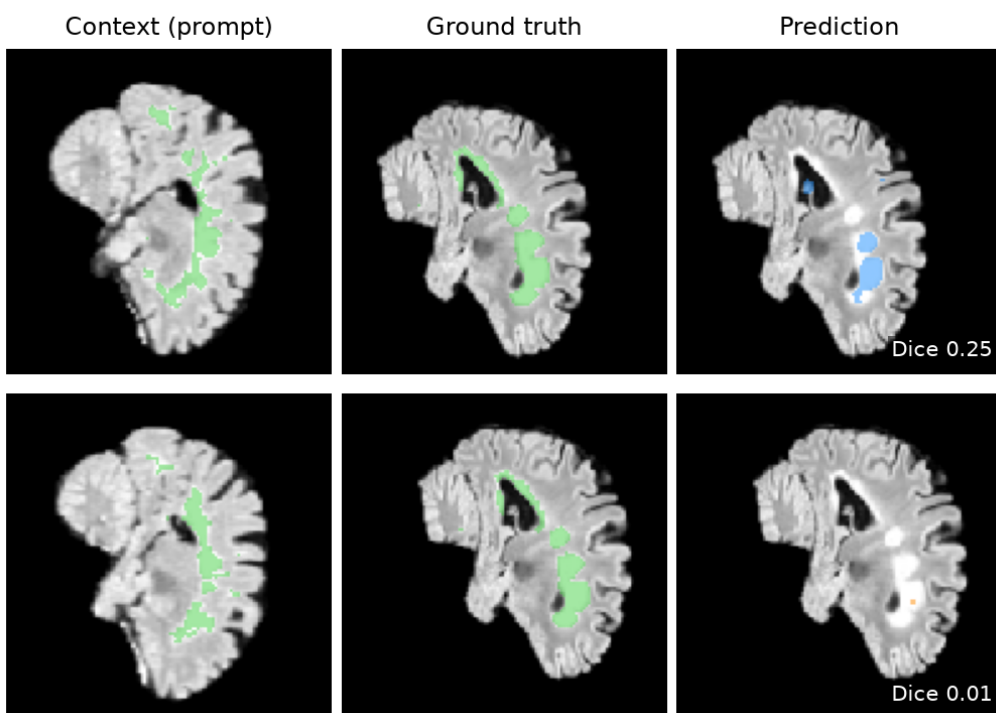


Figure 15: Shifts-MS, example 2.

MSD Hippocampus, hippocampus_anterior: Ours vs. Medverse (subject hippocampus_225)

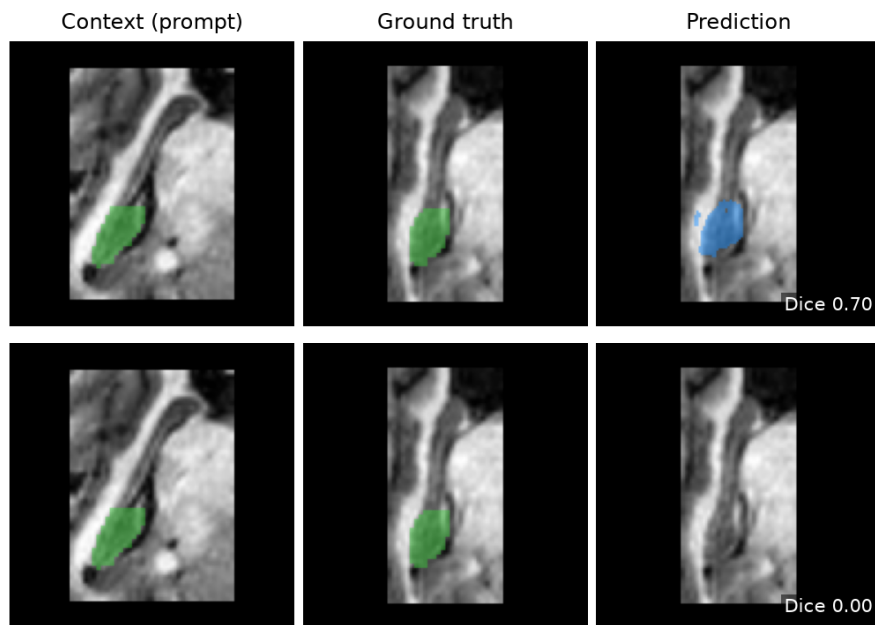


Figure 16: MSD Hippocampus, example 1.

MSD Hippocampus, hippocampus_anterior: Ours vs. Medverse (subject hippocampus_067)

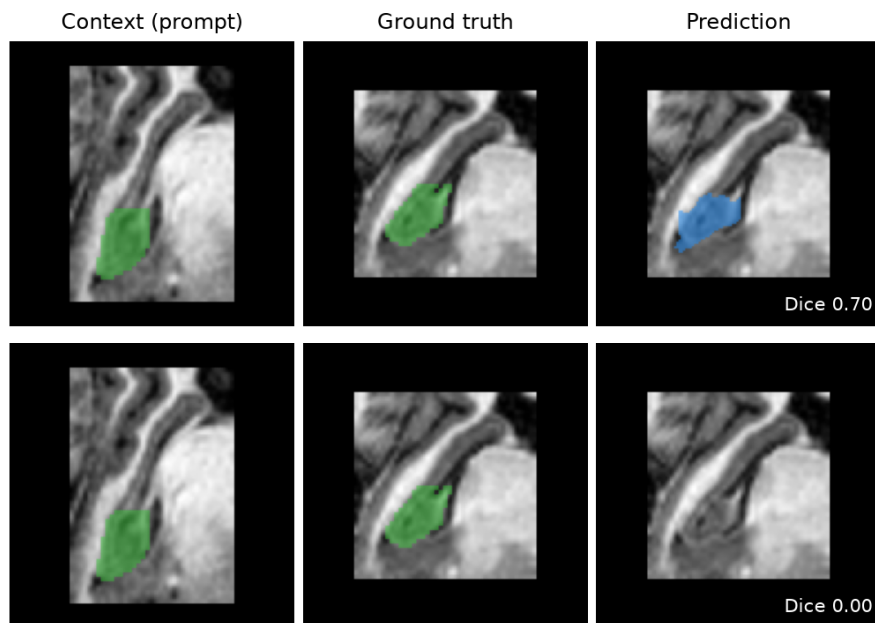


Figure 17: MSD Hippocampus, example 2.

MSD Prostate, prostate_pz: Ours vs. Medverse (subject prostate_18_adc)

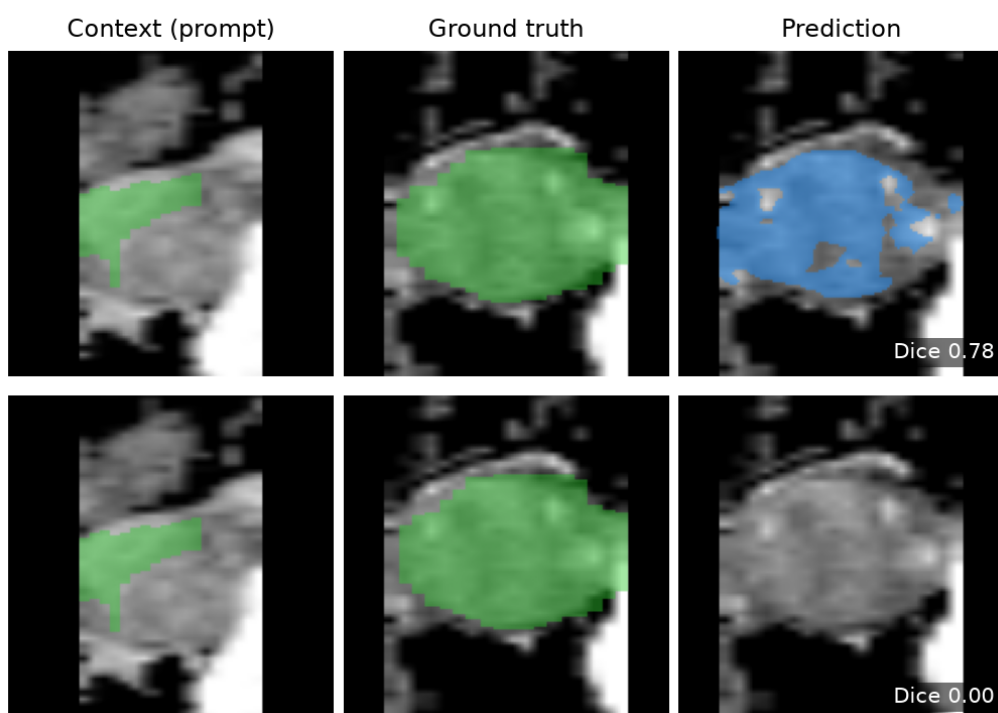


Figure 18: MSD Prostate, example 1.

MSD Prostate, prostate_tz: Ours vs. Medverse (subject prostate_06_adc)

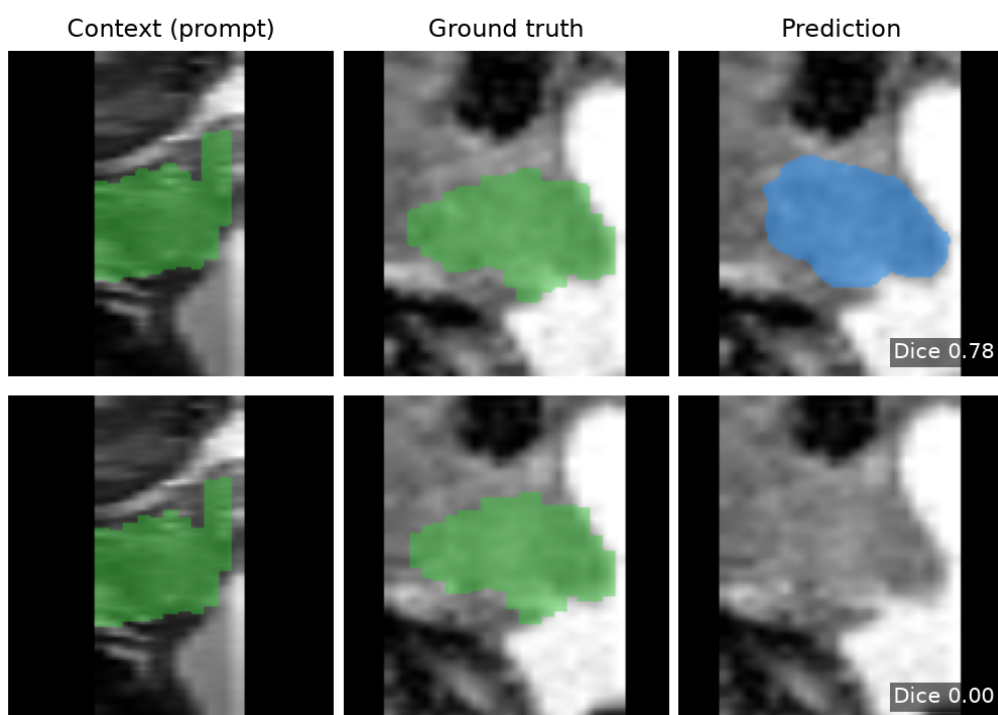


Figure 19: MSD Prostate, example 2.

ATLAS v2.0, stroke_lesion: Ours vs. Medverse (subject sub-r040s020)

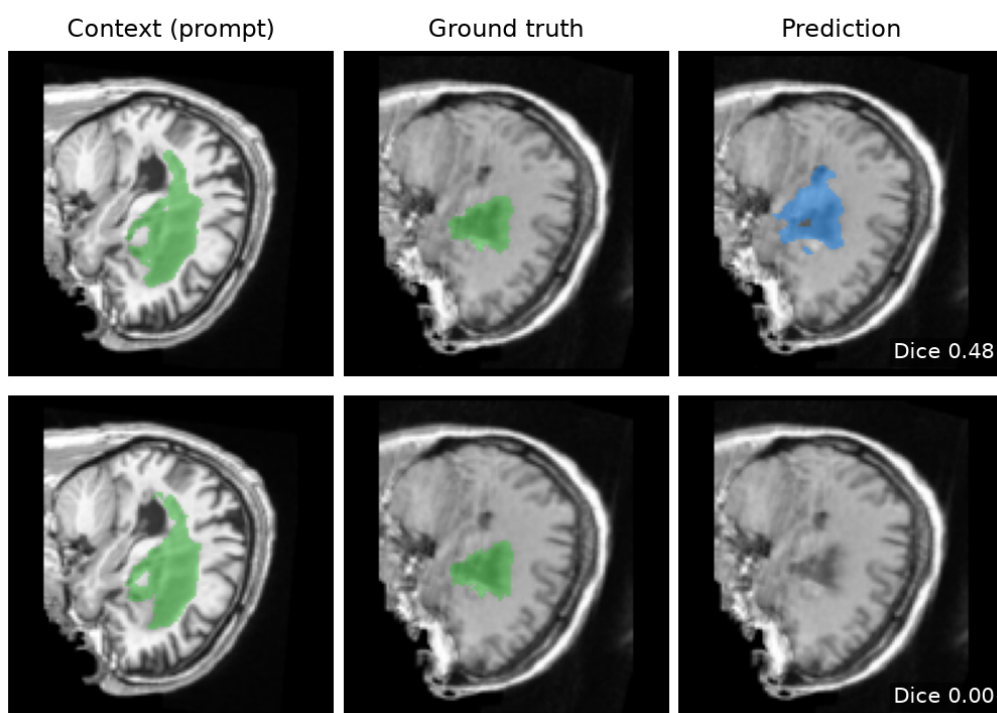


Figure 20: ATLAS v2.0, example 1.

ATLAS v2.0, stroke_lesion: Ours vs. Medverse (subject sub-r027s009)

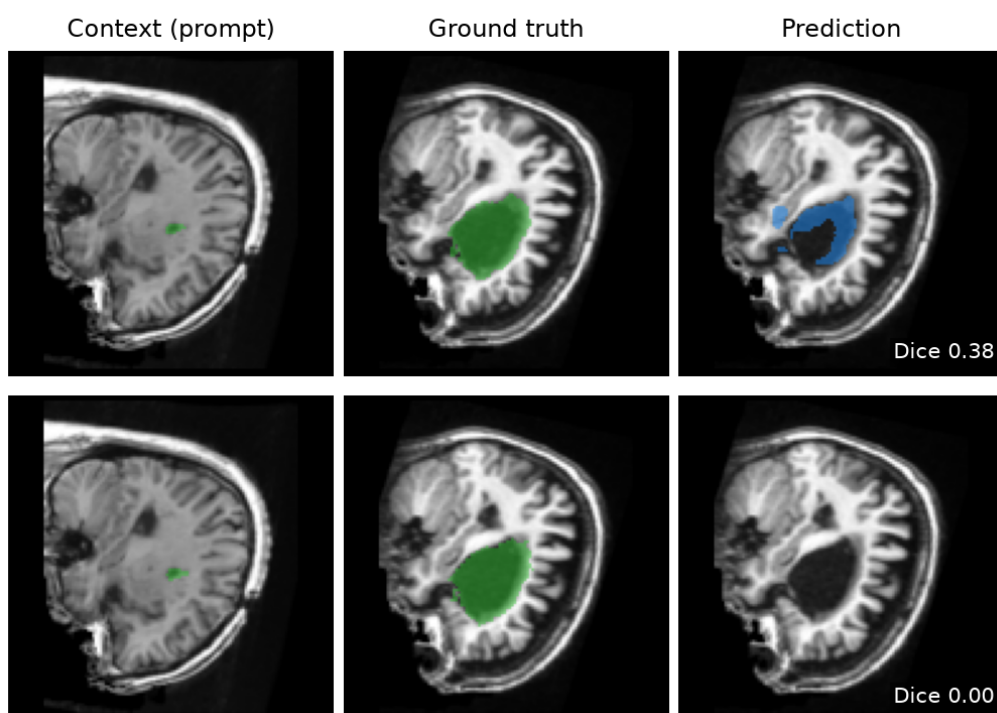


Figure 21: ATLAS v2.0, example 2.

GNC_705, hypo_mask_l: Ours vs. Medverse (subject 117304_v30)

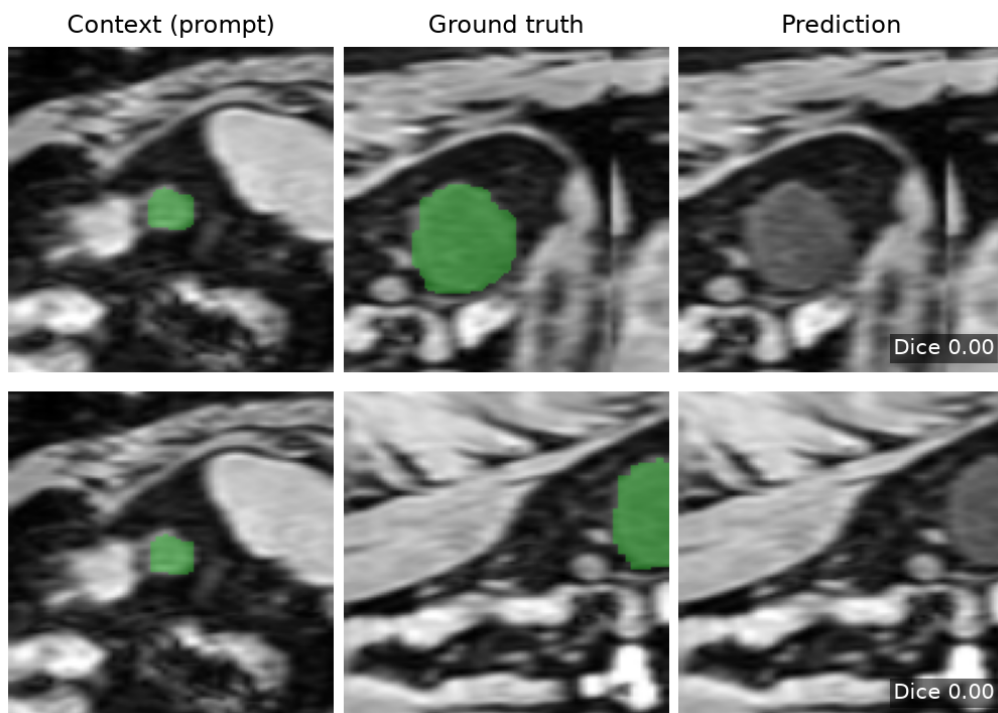


Figure 22: GNC_705, example 1.

GNC_705, complex_cyst_l: Ours vs. Medverse (subject 110703_v30)

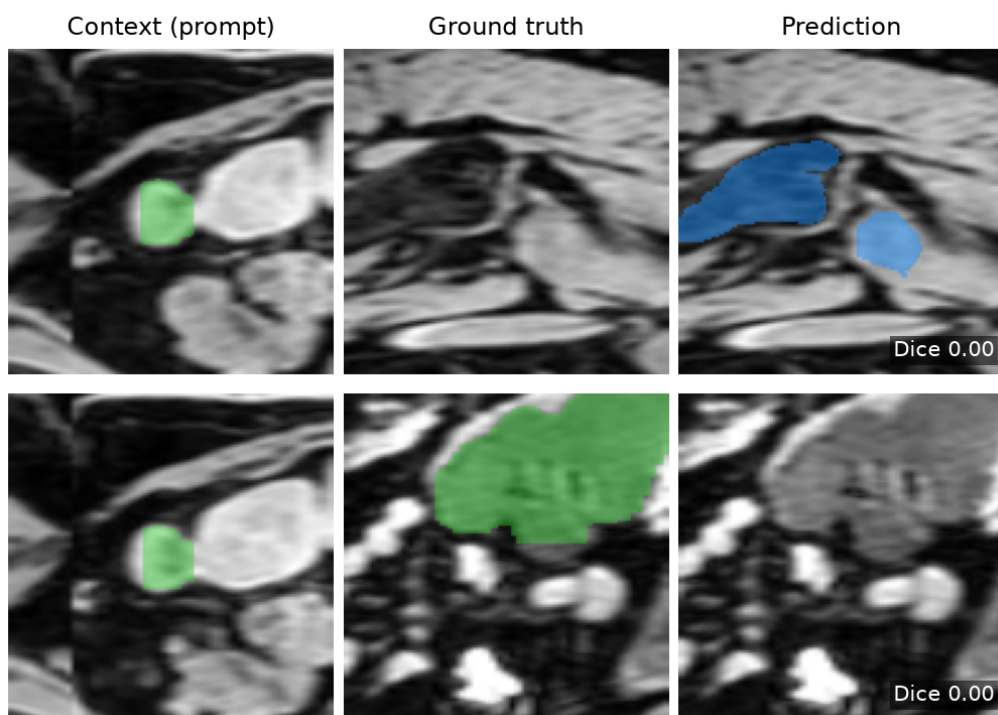


Figure 23: GNC_705, example 2.

FLARE22, spleen: Ours vs. Medverse (subject FLARE22_Tr_0035)

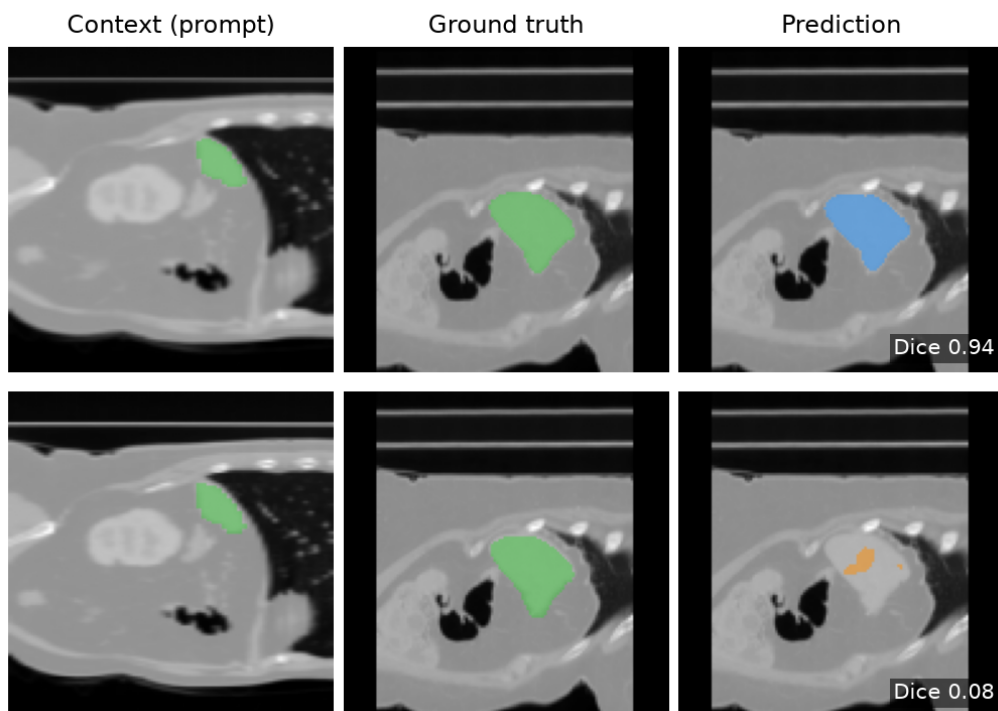


Figure 24: FLARE22, example 1.

FLARE22, gallbladder: Ours vs. Medverse (subject FLARE22_Tr_0039)

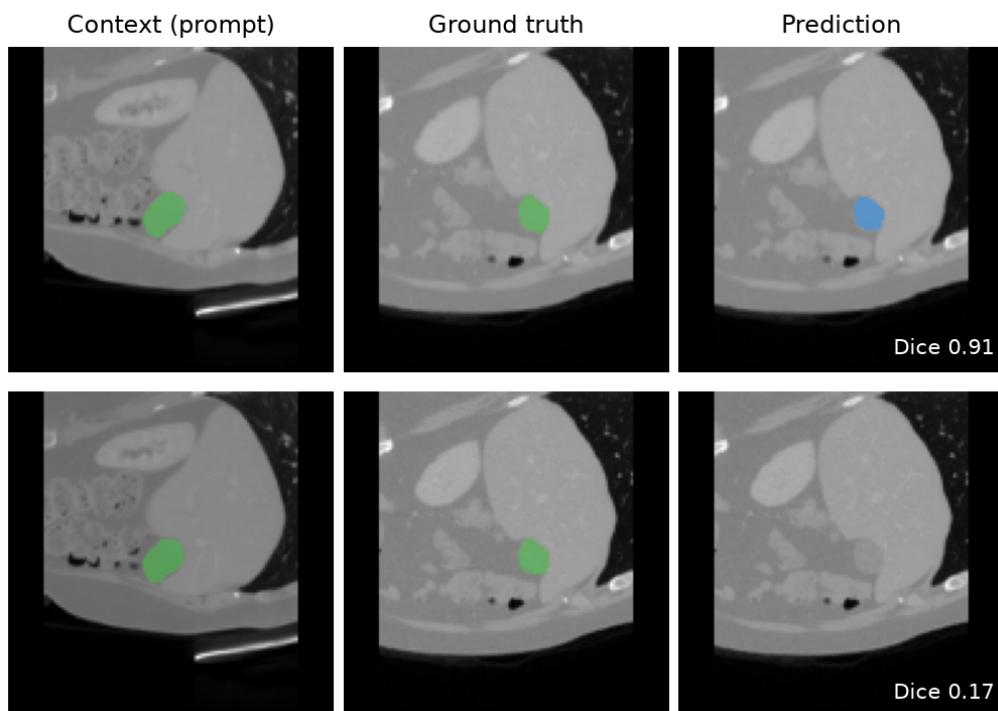


Figure 25: FLARE22, example 2.

NasalSeg, maxillary_sinus_right: Ours vs. Medverse (subject P026)

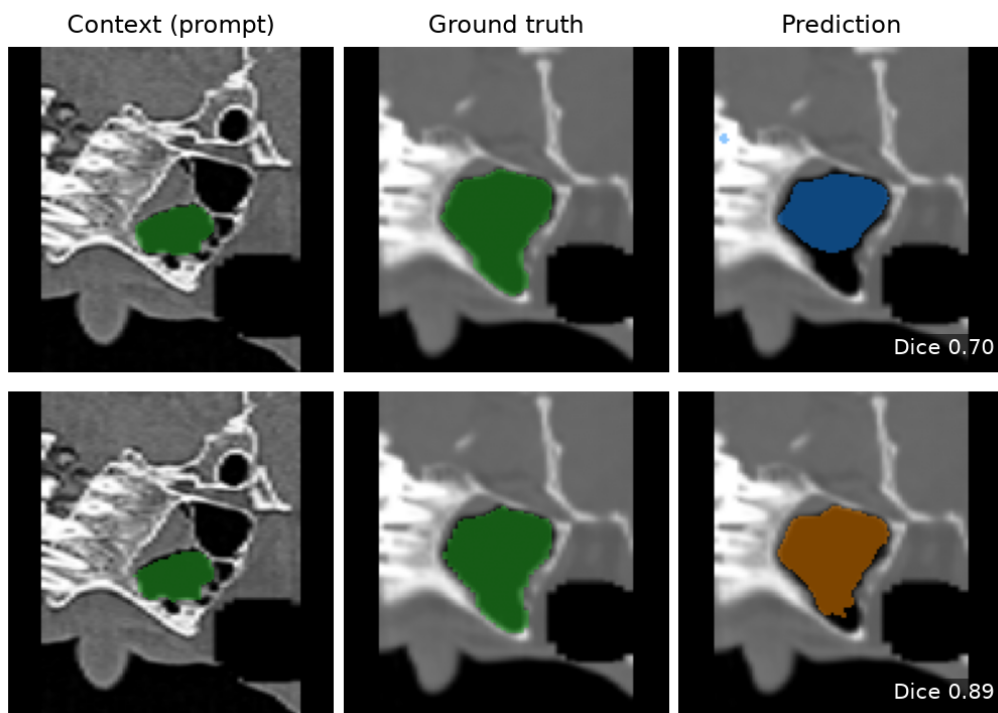


Figure 26: NasalSeg, example 1.

NasalSeg, nasal_cavity_right: Ours vs. Medverse (subject P028)

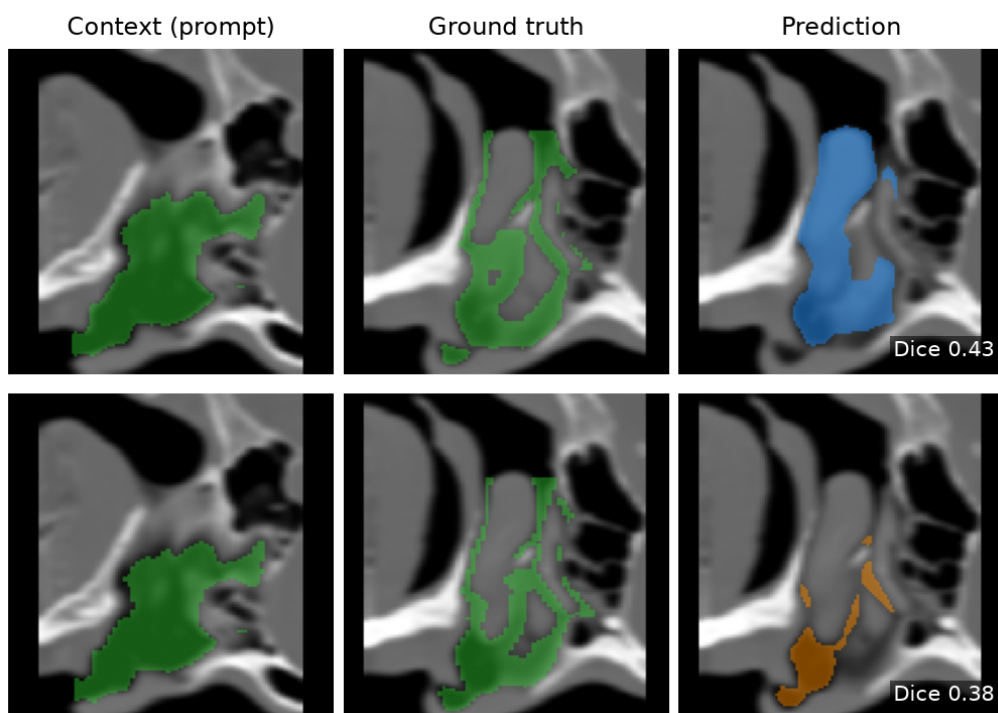


Figure 27: NasalSeg, example 2.

Table 16: Medverse’s released weights run inside our cascade inference (subsection 4.3, “Medverse with our cascade”), with and without its own coarse prediction passed forward as a prior. Macro Dice.

Dataset	No prior	Prior
HU_LWK1	0.000	0.000
ISLES22	0.049	0.035
Shifts-MS	0.036	0.028
MSD Hippocampus	0.691	0.050
MSD Prostate	0.327	0.251
ATLAS v2.0	0.126	0.020
GNC_705	0.008	0.001